



Utopia

Noelia Lázaró Gómez

Prometeo / ThePower

Desarrollo de Aplicaciones Multiplataforma

Luz María Álvarez Moreno

Modalidad Online.

03/05/2026

A mi tutora Luz

*Por darnos esa tranquilidad en la etapa final y por sus ánimos, dándonos ese
impulso que nos falta.*

A mí yo del futuro.

*Por empezar desde cero en un campo que no era el mío.
Este trabajo es la prueba de que el esfuerzo y la determinación pueden cambiar
el rumbo de una vida.*

Índice

<u>Resumen</u>	<u>7</u>
<u>Bloque 1 - Justificación</u>	<u>8</u>
<u>Motivación del proyecto</u>	<u>8</u>
<u>Contexto y Motivación</u>	<u>8</u>
<u>Bloque 2 - Analisis y comparativa</u>	<u>10</u>
<u>Estado de la Cuestión y Comparativa</u>	<u>10</u>
<u>Análisis de Soluciones Existentes.</u>	<u>10</u>
<u>1. Goodreads.</u>	<u>10</u>
<u>2. BookCrossing</u>	<u>11</u>
<u>3. Librología</u>	<u>12</u>
<u>Tabla 1. Comparativa entre apps</u>	<u>14</u>
<u>Conclusión del análisis comparativo</u>	<u>14</u>
<u>Bloque 3 - Funciones y requisitos</u>	<u>14</u>
<u>Introducción</u>	<u>14</u>
<u>Principales funciones y problemas que resuelve</u>	<u>15</u>
<u>Principales requisitos del proyecto</u>	<u>16</u>
<u>Bloque 4 - Objetivos</u>	<u>17</u>
<u>Objetivo general</u>	<u>17</u>
<u>Objetivos Específicos</u>	<u>17</u>
<u>Requisitos del Sistema (RFTP)</u>	<u>18</u>
<u>R01 – Gestión de Acceso y Seguridad</u>	<u>18</u>
<u>R02 – Consulta de Catálogo Bibliográfico</u>	<u>19</u>
<u>R03 – Personalización y Comodidad del Lector (UX)</u>	<u>21</u>
<u>R04 – Gamificación y Automatización de Datos</u>	<u>22</u>

<u>Bloque 5 - Análisis y especificación de requisitos</u>	<u>23</u>
<u>Requisitos funcionales</u>	<u>23</u>
<u>Requisitos no funcionales (RNF)</u>	<u>24</u>
<u>Matriz de Trazabilidad (Objetivos vs. Requisitos)</u>	<u>24</u>
<u>Casos de uso</u>	<u>27</u>
<u>CU-01 — Registrarse en la plataforma</u>	<u>27</u>
<u>CU-02 — Iniciar Sesión</u>	<u>30</u>
<u>CU-03 — Gestionar biblioteca personal</u>	<u>31</u>
<u>CU-04 — Marcar libro disponible</u>	<u>33</u>
<u>CU-05 — Ver libros disponibles de otros usuarios</u>	<u>35</u>
<u>CU-06 — Consultar perfil y logros</u>	<u>37</u>
<u>Bloque 6 - Propuesta de solución</u>	<u>39</u>
<u>Arquitectura del sistema</u>	<u>39</u>
<u>Modelo de datos</u>	<u>41</u>
<u>Relación de Intercambio (Economía Circular)</u>	<u>41</u>
<u>Lógica de backend: triggers en PostgreSQL</u>	<u>42</u>
<u>Sistema de Gamificación y Logros de Perfil</u>	<u>42</u>
<u>Sistema de autenticación con Firebase.</u>	<u>43</u>
<u>Integración con la API de Open Library</u>	<u>44</u>
<u>Tecnologías utilizadas</u>	<u>45</u>
<u>Bloque 7 - Plan de trabajo</u>	<u>47</u>
<u>Planificación</u>	<u>48</u>
<u>Diagrama de Gantt</u>	<u>48</u>
<u>Planificación real + desviaciones.</u>	<u>48</u>
<u>Bloque 8 - Desarrollo de la solución</u>	<u>50</u>

<u>Estructura del proyecto Flutter</u>	<u>50</u>
<u>Implementación funcionalidades</u>	<u>54</u>
<u>Sistema de temas dinámicos (modo claro/oscuro)</u>	<u>54</u>
<u>Seguridad reactiva: AuthGate</u>	<u>57</u>
<u>CRUD de libros con Supabase</u>	<u>59</u>
<u>Integración Open Library</u>	<u>61</u>
<u>Arquitectura de datos del perfil</u>	<u>62</u>
<u>Sistema de logros — Badge Eco Héroe</u>	<u>63</u>
<u>Avatar generado dinámicamente</u>	<u>64</u>
<u>Tests unitarios</u>	<u>64</u>
<u>Triggers PostgreSQL (gamificación)</u>	<u>65</u>
<u>Bloque 9 - Despliegue, pruebas e instalación.</u>	<u>66</u>
<u>Despliegue e instalación</u>	<u>66</u>
<u>Entorno de desarrollo y ejecución en dispositivo físico</u>	<u>66</u>
<u>Generación del APK para instalación independiente</u>	<u>67</u>
<u>Row Level Security (RLS) — Estado actual y plan de activación</u>	<u>67</u>
<u>Pruebas funcionales de backend</u>	<u>69</u>
<u>Pruebas de autenticación</u>	<u>70</u>
<u>Bloque 10 - Evolución y trabajo futuro</u>	<u>71</u>
<u>RF pendiente — Flujo completo de confirmación de intercambio</u>	<u>71</u>
<u>Chat interno en tiempo real para coordinación de intercambios</u>	<u>72</u>
<u>Expansión del ecosistema: Roles para bibliotecas y editoriales</u>	<u>72</u>
<u>Geolocalización y Mapas</u>	<u>73</u>
<u>Tabla 9.Funcionalidades futuras y su estimación.Elaboración propia</u>	<u>73</u>
<u>Bibliografía.</u>	<u>73</u>

Resumen

Utopía nace como respuesta a la necesidad de revitalizar el intercambio cultural de libros físicos mediante tecnología. La plataforma conecta lectores a través de un modelo de economía circular, permitiendo gestionar bibliotecas personales y coordinar intercambios de forma gratuita. La aplicación está desarrollada con Flutter y Dart, garantizando compatibilidad multiplataforma. El resultado es un MVP funcional que cubre el ciclo completo desde el registro del usuario hasta la visualización de libros disponibles para intercambio, con un sistema de gamificación automatizado mediante triggers en PostgreSQL. En definitiva, Utopía es un espacio de diseño ético donde la cultura recupera su lugar, la gestión de datos en un vínculo social humano, sostenible y profundo.

Palabras clave: Flutter, intercambio de libros, economía circular, gamificación, multiplataforma

Abstract

Utopía is a cross-platform mobile application designed to revitalize the physical book exchange culture through technology. The platform connects readers through a circular economy model, allowing them to manage personal libraries and coordinate book exchanges for free. The application is built with Flutter and Dart, ensuring cross-platform compatibility. The result is a functional MVP that covers the complete cycle from user registration to the visualization of books available for exchange, with an automated gamification system implemented through PostgreSQL triggers. Keywords: Flutter, Book exchange, circular economy, gamification, cross-platform

Bloque 1 - Justificación

Motivación del proyecto

Contexto y Motivación

En la era de la digitalización, la lectura en formato físico ha perdido terreno frente a la comodidad de los dispositivos electrónicos. No obstante, el valor tangible de un libro, su tacto, su esencia y la experiencia sensorial que ofrece, sigue siendo insustituible para muchos lectores. El problema actual reside en el coste elevado de las novedades editoriales y las limitaciones de stock en bibliotecas públicas, orientadas mayoritariamente al ámbito académico.

Este proyecto nace de la necesidad de revitalizar el intercambio cultural, utilizando la tecnología para conectar a personas con intereses comunes, permitiendo que los libros físicos sigan circulando y creando vínculos sociales a través de la lectura compartida."

La motivación principal de este proyecto Utopía es fomentar el intercambio "mano a mano", permitiendo que libros con ideas y pensamientos profundos lleguen a lectores que buscan una conexión real. Para el usuario, el valor reside en romper la soledad de la lectura; la plataforma facilita conectar con personas que comparten sus mismos sentimientos e intereses en el mundo físico.

Se propone el desarrollo de una solución multiplataforma de intercambio de libros que permite a los usuarios gestionar su propio catálogo personal mediante un sistema CRUD (Crear, Leer, Actualizar y Borrar). La aplicación no solo facilita el

intercambio físico, sino que actúa como una red social literaria donde los usuarios pueden:

- Crear un perfil profesional con su estantería de lecturas.
- Actualizar la estantería de libros para compartir.
- Borrar libros de la estantería.
- Ver la estantería de otros lectores con los libros disponibles.
- Crear la petición para hacer el intercambio.

Desde una perspectiva tecnológica, el proyecto se motiva en el uso de herramientas de vanguardia para el desarrollo multiplataforma. Se ha seleccionado Flutter ([Google, s.f.-a](#)) como framework y Dart como lenguaje principal. Flutter es reconocida debido a su capacidad para ofrecer una interfaz de usuario (UI) altamente personalizada y fluida en múltiples dispositivos a partir de un único código base, optimizando así los tiempos de desarrollo y mantenimiento.

La arquitectura del proyecto se ha diseñado pensando en la escalabilidad, permitiendo que en futuras versiones se integren nuevos servicios y procesos, para facilitar los encuentros en bibliotecas y puntos de interés cultural.

Bloque 2 - Analisis y comparativa

Estado de la Cuestión y Comparativa

Análisis de Soluciones Existentes.

Para el desarrollo de Utopía, se ha realizado una investigación de las plataformas líderes en el sector literario y de eventos sociales. A continuación, se presenta una comparativa técnica y funcional que justifica la necesidad de nuestra propuesta. Aunque cumplen funciones específicas, presentan carencias que Utopía resuelve de forma integral.

1. Goodreads.

Es la plataforma más grande del mundo para catalogar libros, pero su enfoque ha derivado hacia el marketing editorial (propiedad de Amazon).

- **Ventajas:** Enorme base de datos y comunidad global consolidada. Tiene una gamificación consolidada con sus Readings Challenges anuales que son un estándar en la industria para motivar a los usuarios a leer más.
- **Desventajas:** Interfaz de usuario (UI) anticuada, experiencia de usuario (UX) compleja. Se centra en el “consumismo literario” y en la reseña estática. No ofrece ninguna herramienta para el intercambio físico de libros, obligando al usuario a comprar ejemplares nuevos constantemente.
- **Diferencia con Utopía:** Utopía moderniza la interfaz y prioriza el encuentro físico sobre el registro digital estático. Mientras Goodreads es una biblioteca digital pasiva, Utopía es un motor de economía circular. No

solo gestiona el dato del libro (via API de Open Library), sino que facilita que el objeto físico circule de mano en mano, eliminando el coste económico y el impacto ambiental de la compra de libros nuevos.

2. BookCrossing

BookCrossing es el movimiento global más conocido para “liberar” libros en lugares públicos (cafeterías, parques, bancos...) para que otras personas los encuentren.

- **Ventajas:** Excelente sistema de cultura libre y que los libros viajen sin barreras económicas, con ese factor sorpresa y la experiencia de encontrar un libro en un lugar público que genera una emoción que las plataformas puramente digitales no pueden replicar.
- **Desventajas:** Su tecnología está obsoleta y la experiencia de usuario es frustrante. El intercambio es pasivo, dejas un libro y rezas para que alguien lo encuentre y lo registre. No hay una comunidad real conectada en tiempo real, ni un CRUD que permita ver qué tiene cada vecino en su estantería antes de quedar.
- **Diferencia con Utopía:** Utopía convierte el azar en conexión dirigida. Gracias a la arquitectura sobre Supabase y la API de Open Library, el intercambio es activo y seguro. El usuario no abandona el libro, sino que lo ofrece, generando una petición previa y un encuentro físico que garantiza que el libro llegue a alguien que realmente lo valora. Pasamos del dejar el libro a su suerte a conectar con personas a través de los libros.

3. Librología

App enfocada en el registro y seguimiento de lecturas personales.

Librología es una herramienta centrada exclusivamente en el inventario personal y el seguimiento de hábitos de lectura.

- **Ventajas:** Muy buena para el control individual de la biblioteca del usuario. Ofrecen un entorno libre de distracciones y algoritmos para usuarios que solo buscan organizar su biblioteca sin interactuar con nadie. Podemos añadir sus estadísticas muy detalladas sobre el ritmo de lectura (páginas leídas, páginas leídas por día, tiempo invertido...), funcionando como un entrenador personal para lectores.
- **Desventajas:** Carece de funciones sociales de comunidad. Es una aplicación como se diría comúnmente llamada isla para asociarla a algo aislado de todo lo demás. Aunque es excelente para el control individual de una biblioteca (CRUD personal), carece totalmente de funciones sociales. El usuario registra lo que lee, pero esa información no sale de su dispositivo, no hay flujo de libros ni interacción con otros lectores.
- **Diferencia con Utopía:** Utopía transforma el inventario estático en un ecosistema dinámico. Mientras en Librología el usuario es solo un registrador, en Utopía introducimos roles activos: el usuario es a la vez lector, donante o receptor dentro de una economía circular. Pasamos del

simple control de una biblioteca personal a la gestión de una biblioteca comunitaria, donde el dato del libro sirve para activar un encuentro real entre personas.

Análisis comparativo de soluciones existentes

Utopía frente a las principales plataformas del sector literario

Funcionalidad	Goodreads	BookCrossing	Librología	✦ Utopía
GESTIÓN DE LIBROS				
Catálogo personal de libros CRUD: crear, leer, actualizar, eliminar	✓	X	✓	✓
API de catálogo abierto Datos bibliográficos automáticos	X	X	X	✓
Estadísticas personales Contador de intercambios y logros	~	X	✓	✓
INTERCAMBIO Y COMUNIDAD				
Intercambio físico entre usuarios diferenciador Economía circular mano a mano	X	~	X	✓
Solicitud de intercambio diferenciador El usuario pide un libro a otro lector	X	X	X	✓
Comunidad en tiempo real Ver libros disponibles de otros usuarios	~	X	X	✓
EXPERIENCIA DE USUARIO				
Gamificación y logros Sistema automático Eco Héroe	~	X	X	✓
Modo claro / oscuro Adaptación visual dinámica	X	X	X	✓
Perfil público de usuario Con estantería visible para otros	✓	X	X	✓
Puntos de encuentro Coordinación para intercambio físico	X	X	X	~

✓ Implementado · ~ Parcial o limitado · X No disponible · ✦ Utopía destaca especialmente en el intercambio físico directo entre lectores mediante economía circular.

Tabla 1. Comparativa entre apps

Conclusión del análisis comparativo

A diferencia de las soluciones comerciales analizadas, Utopía se ha diseñado bajo una arquitectura de roles que garantiza su expansión futura. Aunque la versión actual se focaliza en el perfil de lector para asegurar un intercambio fluido, el sistema está preparado estructuralmente para integrar roles de entidad cultural a futuro (librerías y autores independientes).

Esta capacidad permitirá, en fases posteriores, la autogestión de Club de lecturas y eventos de presentación por parte de profesionales del sector. De este modo, la aplicación trasciende el concepto de catálogo estático para convertirse en una agenda cultural activa y un punto de encuentro para la comunidad.

Bloque 3 - Funciones y requisitos

Introducción

Utopía nace para dar respuesta a la creciente necesidad de una comunidad literaria libre de los sesgos de los algoritmos comerciales de las redes sociales generalistas. El proyecto busca devolver el protagonismo al lector y al libro como objeto de intercambio cultural.

Principales funciones y problemas que resuelve

Los principales problemas que resuelve esta plataforma son:

- **Deshumanización del descubrimiento literario:** Sustituye la publicidad intrusiva y los intereses de las grandes editoriales por recomendaciones orgánicas y sinceras basadas en la afinidad real entre lectores.
- **Desuso de recursos :** Resuelve el problema de los libros que acumulan polvo en las estanterías mediante un modelo de economía circular, dándoles una segunda vida a través del intercambio gratuito.
- **Aislamiento del lector:** Rompe la barrera digital facilitando encuentros físicos, transformando la soledad de la lectura en un acto social compartido.
- **Falta de motivación en la lectura:** Combate el abandono del hábito lector mediante un sistema de gamificación, donde el usuario es recompensado por sus acciones dentro de la plataforma.
- **Impacto ambiental y económico:** Resuelve la ineficiencia del consumo lineal de libros mediante un modelo de economía circular, reduciendo la huella de carbono al fomentar el intercambio local y gratuito.

Principales requisitos del proyecto

Para cumplir con estos propósitos, la aplicación se apoya en los siguientes pilares técnicos y funcionales que definen su alcance:

1. **Seguridad y Personalización:** Implementación de un sistema de acceso único por vía Firebase Authentication, para que cada lector gestione su acceso a la plataforma.
2. **Consulta y sincronización en tiempo real:** Integración con la API de Open Library para la obtención instantánea del catálogo siempre actualizado y veraz, evitando que el usuario tenga que escribir manualmente la información de cada libro.
3. **Gestión Dinámica de Datos (CRUD):** Capacidad técnica para crear, leer, actualizar y eliminar registros de libros de forma fluida mediante Supabase, permitiendo una administración total por parte del usuario.
4. **Sistema de Recompensas y Logros (Gamificación):** Implementación de una lógica de "metas" en la base de datos que premia hitos clave, como registrar la primera lectura o completar el primer intercambio. Esto incentiva la participación activa en la economía circular.
5. **Comodidad Visual y Accesibilidad:** Adaptación de la interfaz mediante modos de lectura (claro/oscuro) que respeten la salud visual del usuario y reduce la fatiga visual durante sesiones prolongadas.

Bloque 4 - Objetivos

Objetivo general

Desarrollar una aplicación móvil multiplataforma con Flutter que permita a los usuarios gestionar su biblioteca personal de libros físicos y conectar con otros lectores para facilitar su intercambio, integrando servicios externos de catalogación bibliográfica, autenticación segura en la nube y un sistema de gamificación que incentive la economía circular literaria.

Objetivos Específicos

Para alcanzar el objetivo general, se han definido las siguientes metas técnicas y funcionales:

- **O.E.1. Implementar un sistema de gestión de catálogo (CRUD):**
Permitir al usuario dar de alta, consultar, modificar y eliminar ejemplares de su biblioteca personal de forma dinámica.
- **O.E.2. Integrar servicios de información bibliográfica:** Automatizar la obtención de datos (títulos, autores y portadas) mediante la conexión con la **API externa de Open Library**.
- **O.E.3. Garantizar la seguridad y persistencia de datos:** Configurar un sistema de autenticación de usuarios con **Firebase** y una base de datos en la nube con **Supabase** para asegurar la integridad de la información.

- **O.E.4. Desarrollar un sistema de gamificación:** Programar lógica de backend (Triggers en PostgreSQL) que reconozca hitos de usuario y otorgue logros automáticos para incentivar el intercambio.
- **O.E.5. Optimizar la experiencia de usuario (UX/UI):** Diseñar una interfaz fluida en **Flutter** que incluya adaptación a modos visuales (claro/oscuro) para mejorar la accesibilidad.
- **O.E.6. Habilitar el intercambio entre usuarios:** Permitir que los usuarios publiquen sus libros como disponibles y consulten los catálogos de otros lectores para facilitar el intercambio físico.

Requisitos del Sistema (RFTP)

Siguiendo la metodología exigida, se presenta el desglose técnico de los objetivos principales implementados en esta fase del proyecto.

R01 – Gestión de Acceso y Seguridad

El sistema debe permitir el registro y login seguro de usuarios

- **F01 – Autenticación de usuarios:** Autenticación de usuarios con Firebase
 - **T01:** Implementar el servicio de Firebase Authentication en el proyecto Flutter.
 - **P01:** Registro de un nuevo usuario desde la interfaz y verificación en la consola de Firebase.

- **F02 - Sincronización Supabase:** Sincronización automática con Supabase.
 - **F02T01:** Implementar el componente AuthGate con StreamBuilder ([Google, s.f.-d.](#))
 - **F02P01:** Verificar redirección automática según estado de sesión.



Figura 1. Diagrama RFTP del requisito R01 — Elaboración propia mediante Napkin.ai.

R02 – Consulta de Catálogo Bibliográfico

El sistema debe conectar con Open Library para obtener datos de libros automáticamente.

- **F01 – Integración de API Externa:** Conexión con Open Library mostrando libros en tendencia en la pantalla principal.
 - **T01:** Configurar las peticiones HTTP al endpoint de tendencias.

- **P01:** Visualizar títulos y portadas reales en la pantalla de inicio al iniciar la app.
- **F02 - Carga automática de portada por ISBN**
 - **F02T01:** Construir URL de portada desde ISBN almacenado en Supabase.
 - **F02P01:** Añadir libro con ISBN y verificar que aparece la portada real.

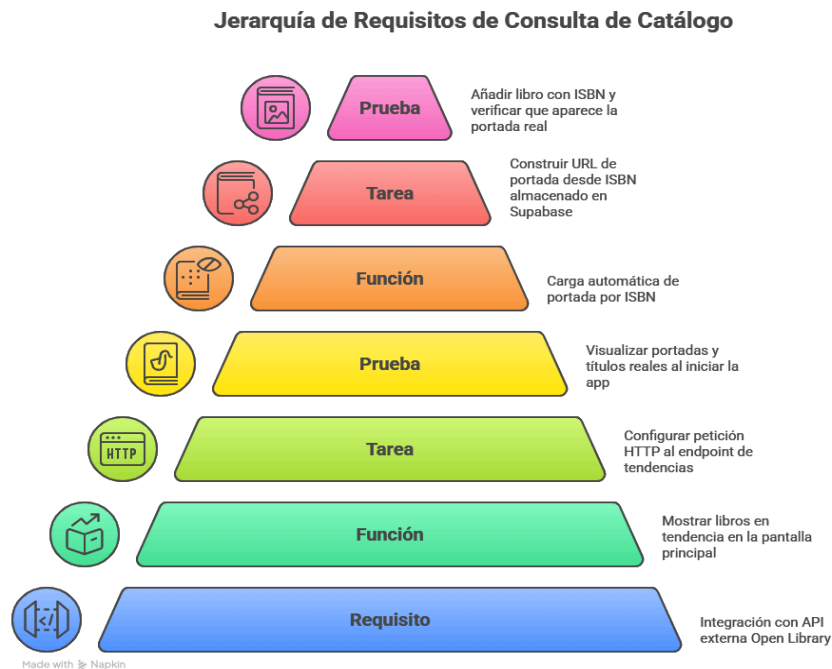


Figura 2. Diagrama RFTP del requisito R02 — Elaboración propia mediante Napkin.ai.

R03 – Personalización y Comodidad del Lector (UX)

- **F01 – Soporte para Modo Oscuro/Claro:** Sistema de temas dinámicos con ThemeProvider.

- **T01:** Implementar *ThemeProvider* ([Google.. s.f.-c.](#)) para gestionar el estado del tema globalmente.
- **P01:** Activar el Switch en el menú lateral (drawer) y verificar el cambio de colores instantáneo.

Requisitos de Personalización Visual

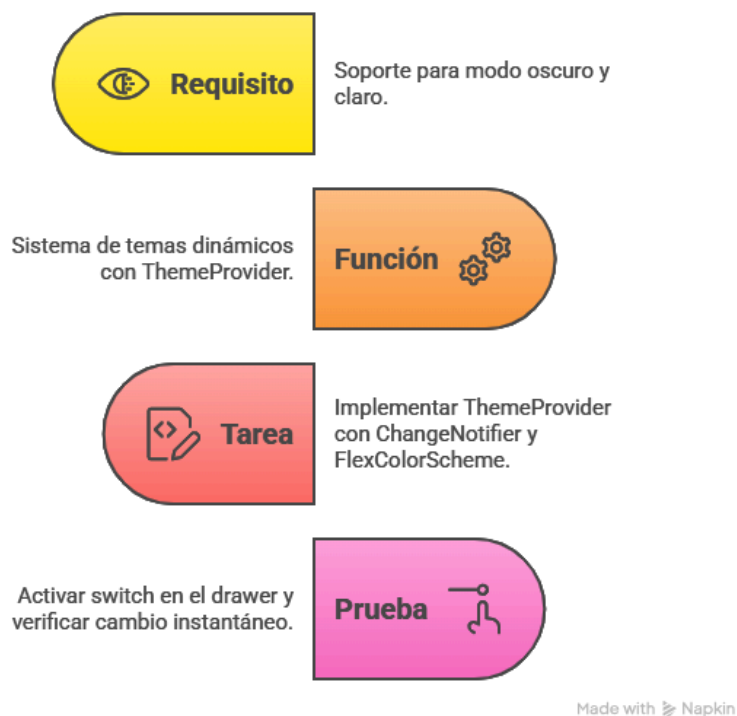


Figura 3. Diagrama RFTP del requisito R03 — Elaboración propia mediante Napkin.ai.

R04 – Gamificación y Automatización de Datos

- **F01 – Estatus Dinámico de Usuario:** El sistema debe reconocer y recompensar automáticamente a los usuarios activos en la plataforma.

- **T04: Implementación de Triggers en Supabase:** Programar un disparador (*trigger*) en el backend que evalúe el volumen de intercambios y actualice el rango de "Eco Héroe" sin intervención.
- **P01: Verificación:** Modificar el contador de libros en la base de datos (`books_exchanged`) y verificar que el estatus cambia a verdadero (`is_eco_hero = true`) de forma inmediata.



Figura 4. Diagrama RFTP del requisito R04 — Elaboración propia mediante Napkin.ai.

Bloque 5 - Análisis y especificación de requisitos

Requisitos funcionales

- **RF-01:** El sistema permitirá el registro y login de usuarios con Firebase.

- **RF-02:** El sistema mostrará libros en tendencia semanal mediante la API de Open Library en la pantalla principal.
- **RF-03:** El usuario podrá gestionar su biblioteca personal mediante las operaciones de creación, consulta, edición y eliminación de ejemplares (CRUD).
- **RF-04:** El sistema otorgará logros automáticos según la actividad (Eco Héroe).
- **RF-05:** El usuario podrá marcar libros de su biblioteca como "disponibles para intercambio" .
- **RF-06:** El usuario podrá visualizar los libros disponibles de otros usuarios registrados en la plataforma.

Requisitos no funcionales (RNF)

- **RNF-01 (Usabilidad):** La interfaz debe ser intuitiva y contar con Modo Oscuro/Claro.
- **RNF-02 (Multiplataforma):** La aplicación debe ejecutarse en Android e iOS (Flutter).

- **RNF-03 (Persistencia):** Los datos deben guardarse de forma segura en la nube (Supabase).
- **RNF-04 (Rendimiento):** Las consultas a la API no deben bloquear la interfaz de usuario .
- **RNF-05 (Seguridad):** El acceso a los datos de usuario debe estar protegido mediante autenticación verificada. Ningún usuario podrá acceder ni modificar registros que no le pertenezcan.

Matriz de Trazabilidad (Objetivos vs. Requisitos)

Para garantizar la coherencia del proyecto, se presenta la siguiente matriz donde cada **requisito funcional** (RF) y **no funcional** (RNF) da respuesta directa a los **objetivos específicos** (OE) definidos en el apartado anterior y con su estimación.

Código Requisito	Descripción del Requisito	Tipo	Objetivo Relacionado	Estimación
RF-01	El sistema permitirá el registro y login mediante Firebase.	Funcional	O.E.3	Alta - 3–4 días.

RF-02	El sistema mostrará libros en tendencia semanal mediante la API de Open Library en la pantalla principal.	Funcional	O.E.2	Media - 2 días.
RF-03	Alta, consulta, edición y borrado de libros (CRUD).	Funcional	O.E.1	Alta - 4-5 días.
RF-04	Sistema automático de logros y rangos (Eco Héroe).	Funcional	O.E.4	Media - 2 días.
RF-05	El usuario podrá marcar libros de su biblioteca como disponibles para intercambio	Funcional	O.E.6	Baja - 1 día.
RF-06	Ver libros de otros usuarios	Funcional	O.E.6	Baja - 1 día.
RNF-01	La aplicación debe ejecutarse en Android e iOS	No Func.	O.E.5	— (inherente a Flutter)

RNF-02	Persistencia de datos en la nube con Supabase.	No Func.	O.E.3	Media - 2 días
RNF-03	Interfaz adaptable con Modo Oscuro/Claro.	No Func.	O.E.5	Baja - 1 día
RNF-04	Las consultas a la API no deben bloquear la interfaz de usuario.	No Func.	O.E.5	Baja - 1 día
RNF-05	El acceso a los datos debe estar protegido mediante autenticación. Ningún usuario podrá acceder ni modificar registros ajenos.	No Func.	O.E.3	Alta - 2 a 3 días

Tabla 2. Matriz de trazabilidad entre objetivos específicos y requisitos del sistema. Elaboración propia.

Casos de uso

Se presentan a continuación los seis casos de uso principales del sistema, seleccionados por representar los flujos de mayor impacto funcional para el usuario: autenticación, gestión del catálogo, intercambio y perfil.

CU-01 — Registrarse en la plataforma

Este caso de uso describe el proceso de alta de un nuevo usuario, garantizando la persistencia tanto en el sistema de autenticación como en la base de datos de perfiles.

Descripción	El usuario crea una cuenta nueva mediante correo electrónico y contraseña
Actor	Usuario no registrado
Precondiciones	El usuario no dispone de cuenta previa en Firebase
Postcondiciones	Se crea el usuario en Firebase Authentication y se inserta su perfil en la tabla profiles de Supabase mediante upsert

Datos de entrada	Correo electrónico, contraseña
Datos de salida	UID de usuario, token de sesión activa
Flujo principal	1. Usuario accede a RegisterScreen 2. Introduce credenciales 3. Sistema llama a AuthService.signUp() 4. Firebase crea el usuario 5. AuthGate detecta sesión activa 6. Sistema ejecuta _syncUserWithSupabase() 7. Usuario accede a HomeScreen
Flujo alternativo	Si el correo ya existe, Firebase devuelve error y la app muestra SnackBar con mensaje de error
Tablas afectadas	profiles (Supabase)
Clases involucradas	RegisterScreen, AuthService,

	AuthGate
--	----------

Tabla 3. Caso de uso CU-01 — Registrarse en la plataforma. Elaboración propia.

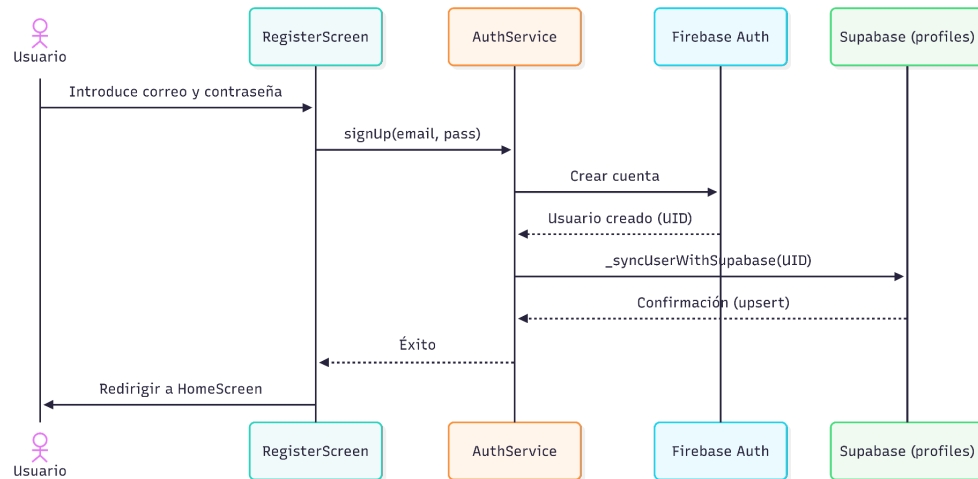


Figura 5. Diagrama de secuencia del caso de uso CU-01 — Registrarse en la plataforma.

CU-02 — Iniciar Sesión

Descripción	El usuario accede a la plataforma introduciendo sus credenciales registradas
Actor	Usuario registrado
Precondiciones	El usuario dispone de cuenta previa en Firebase Authentication

Postcondiciones	Sesión activa. AuthGate redirige automáticamente a HomeScreen
Datos de entrada	Correo electrónico, contraseña
Datos de salida	Token de sesión activa, redirección a HomeScreen
Flujo principal	1. Usuario accede a LoginScreen 2. Introduce email y contraseña 3. Sistema llama a AuthService.signIn() 4. Firebase valida las credenciales 5. AuthGate detecta sesión activa mediante authStateChanges() 6. Sistema redirige automáticamente a HomeScreen
Flujo alternativo	Si las credenciales son incorrectas, Firebase devuelve error y la app muestra SnackBar con mensaje de error. El usuario permanece en LoginScreen
Tablas afectadas	profiles (solo lectura para verificar sincronización)
Clases involucradas	LoginScreen, AuthService, AuthGate

Tabla 4. Caso de uso CU-02 — Iniciar sesión en la plataforma.

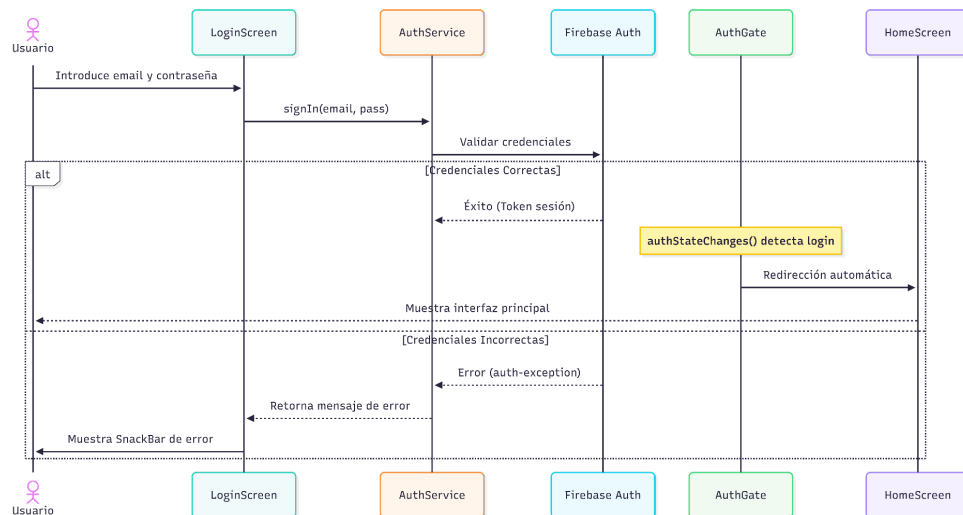


Figura 6. Diagrama de secuencia del caso de uso CU-02 — Iniciar sesión en la plataforma.

CU-03 — Gestionar biblioteca personal

Descripción	El usuario puede añadir, consultar, editar el estado y eliminar libros de su catálogo personal
Actor	Usuario autenticado
Precondiciones	Sesión activa en Firebase
Postcondiciones	Los cambios persisten en la tabla books_circulation de Supabase vinculados al UID del usuario
Datos de entrada	Título del libro, ISBN (opcional)

Datos de salida	Lista actualizada de libros con portadas cargadas desde Open Library
Flujo principal	1. Usuario accede a BooksScreen [5] 2. Pulsa añadir libro 3. Introduce título e ISBN 4. BookRepository ejecuta INSERT en Supabase 5. Si hay ISBN, BookCard construye URL de portada automáticamente 6. Lista se refresca con FutureBuilder
Flujo alternativo	Si no hay ISBN, se muestra placeholder con esquema Sakura
Tablas afectadas	books_circulation (Supabase)
Clases involucradas	BooksScreen, BookRepository, BookModel, BookCard

Tabla 5. Caso de uso CU-03 — Gestionar biblioteca personal.

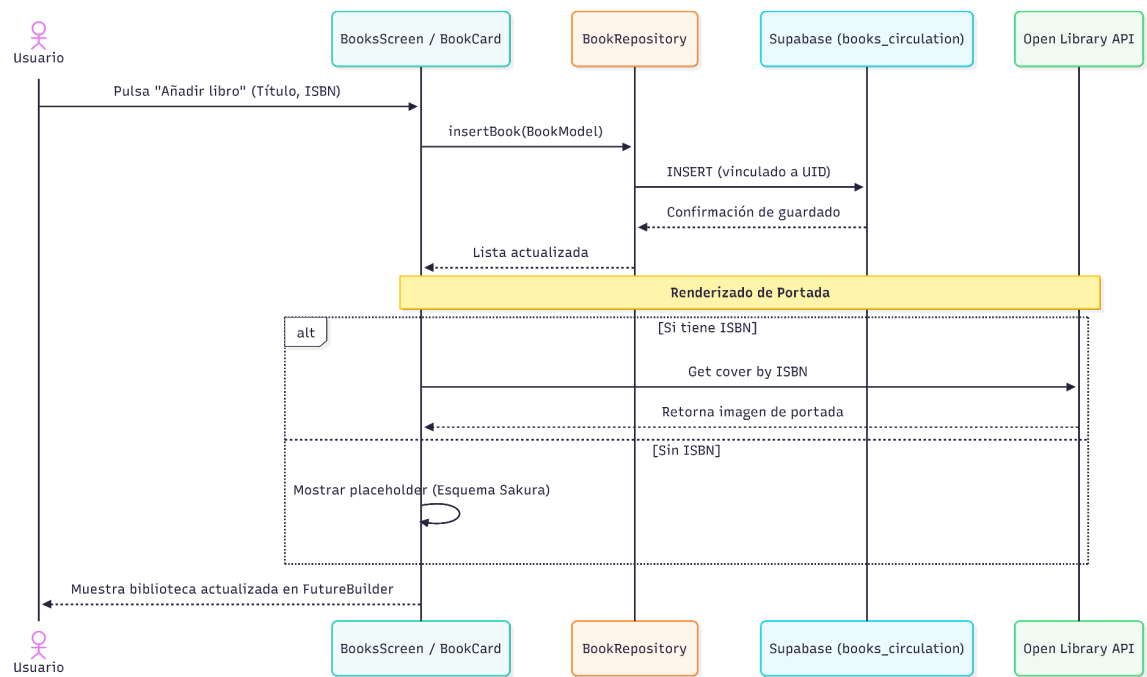


Figura 7. Diagrama de secuencia del caso de uso CU-03 — Gestionar biblioteca personal.

CU-04 — Marcar libro disponible

Descripción	El usuario cambia el estado de un libro de su biblioteca para ofrecerlo a otros lectores
Actor	Usuario autenticado
Precondiciones	Sesión activa. El libro existe en la tabla books_circulation vinculado al UID del usuario

Postcondiciones	El campo status del libro se actualiza a disponible en Supabase. El libro es visible para otros usuarios
Datos de entrada	ID del libro, nuevo estado (disponible / en mi estantería)
Datos de salida	Estado actualizado reflejado en la interfaz mediante badge
Flujo principal	1. Usuario accede a BooksScreen 2. Localiza el libro en su lista 3. Pulsa el botón de cambio de estado en BookCard 4. BookRepository ejecuta UPDATE en Supabase 5. La interfaz refresca el badge de estado automáticamente
Flujo alternativo	Si la actualización falla por error de red, se muestra Snackbar de error y el estado visual no cambia
Tablas afectadas	books_circulation (campo status)
Clases involucradas	BooksScreen, BookCard, BookRepository

Tabla 6. Caso de uso CU-04 — Marcar libros disponibles

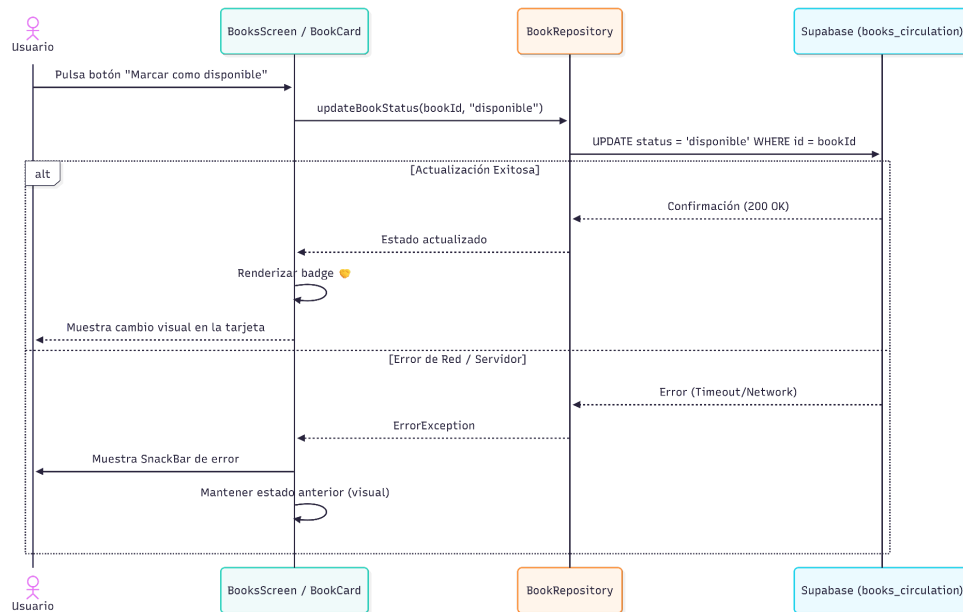


Figura 8. Diagrama de secuencia del caso de uso CU-04 — Registrarse en la plataforma.

CU-05 — Ver libros disponibles de otros usuarios

Descripción	El usuario puede explorar los libros que otros lectores han marcado como disponibles para intercambio
Actor	Usuario autenticado
Precondiciones	Sesión activa. Existen registros en books_circulation con status = disponible de otros usuarios
Postcondiciones	Ninguna. Operación de solo lectura

Datos de entrada	UID del usuario autenticado (para excluir sus propios libros del listado)
Datos de salida	Lista de libros con título, portada y nombre del propietario actual
Flujo principal	1. Usuario navega a la sección de intercambio 2. Sistema consulta books_circulation filtrando status = disponible y excluyendo los libros del propio usuario 3. Se renderizan las tarjetas con portada cargada desde Open Library por ISBN 4. El usuario puede mandar una petición al propietario para coordinar el intercambio
Flujo alternativo	Si no hay libros disponibles, se muestra mensaje informativo invitando al usuario a marcar sus propios libros
Tablas afectadas	books_circulation (solo lectura), profiles (para obtener nombre del propietario)
Clases involucradas	ExchangeScreen, BookCard, BookRepository

Tabla 7. Caso de uso CU-05 — Ver libros disponibles de otros usuarios.

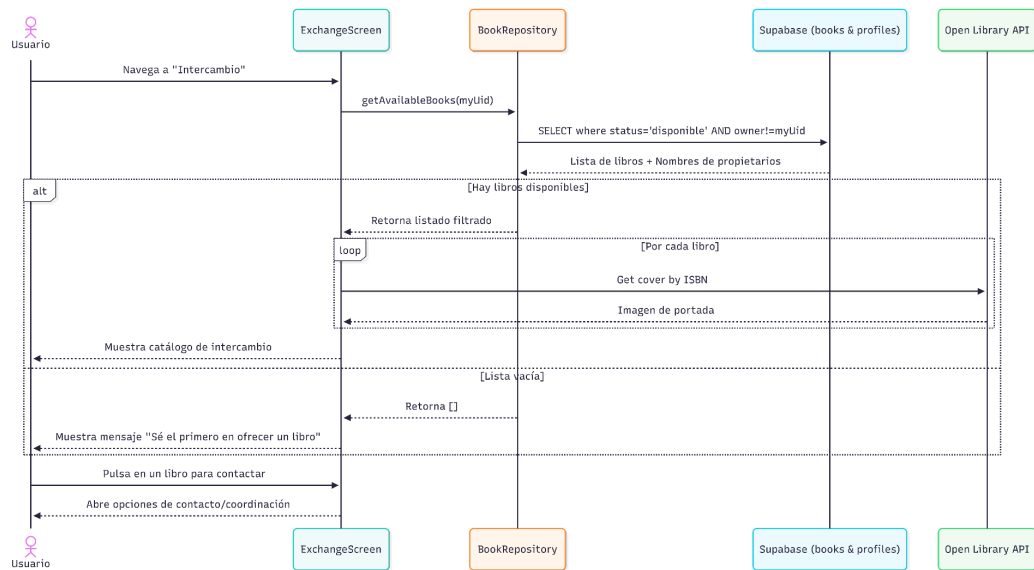


Figura 9. Diagrama de secuencia del caso de uso CU-5 — *Ver libros disponibles de otros usuarios.*

CU-06 — Consultar perfil y logros

Descripción	El usuario consulta su perfil, estadísticas de actividad y estado del logro Eco Héroe
Actor	Usuario autenticado
Precondiciones	Sesión activa. Perfil existente en tabla profiles
Postcondiciones	Ninguna. Operación de solo lectura
Datos de entrada	UID de Firebase (obtenido automáticamente)

Datos de salida	Nombre de usuario, email, contador de intercambios, estado booleano is_eco_hero
Flujo principal	1. Usuario abre ProfileScreen desde DrawersOne 2. Se ejecuta _loadProfile() consultando Supabase por UID 3. Si is_eco_hero = true, se renderiza el badge Eco Héroe 4. Avatar se genera dinámicamente desde la inicial del nombre
Flujo alternativo	Si el perfil no carga, se muestra estado vacío con valores por defecto
Tablas afectadas	profiles (solo lectura)
Clases involucradas	ProfileScreen, DrawersOne

Tabla 8. Caso de uso CU-06 — Consultar perfil y logros.

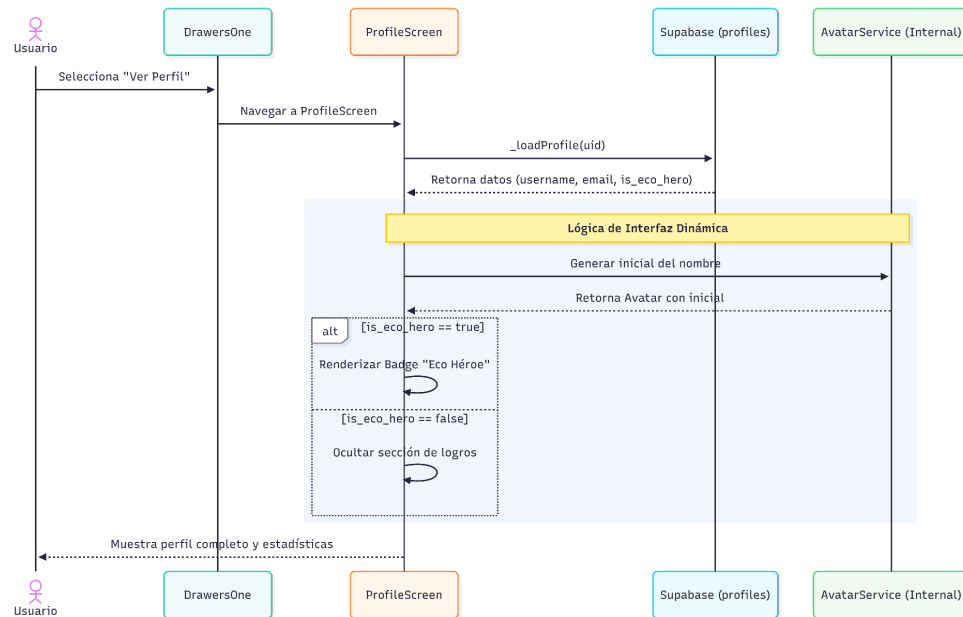


Figura 10. Diagrama de secuencia del caso de uso CU-6 — Consultar perfil y logros.

Bloque 6 - Propuesta de solución

Arquitectura del sistema

La arquitectura de Utopía sigue un modelo cliente-servidor desacoplado de **tres capas**. La capa de presentación está implementada íntegramente en Flutter con Dart, generando un único código base que se compila de forma nativa tanto para Android como para iOS. La capa de servicios se divide en dos proveedores externos especializados: Firebase Authentication gestiona el ciclo completo de identidad del usuario (registro, inicio de sesión y cierre de sesión), mientras que Supabase actúa como backend principal, ofreciendo una base de datos PostgreSQL en la nube con capacidad de consulta en tiempo real. La capa de

datos externos se completa con la integración de la API pública de Open Library, que proporciona los metadatos bibliográficos (título, autor, portada e ISBN) de forma automática, eliminando la necesidad de introducción manual por parte del usuario.

Esta separación de responsabilidades permite que cada servicio pueda evolucionar o sustituirse de forma independiente sin afectar al resto del sistema, dotando a la arquitectura de escalabilidad y mantenibilidad a largo plazo.

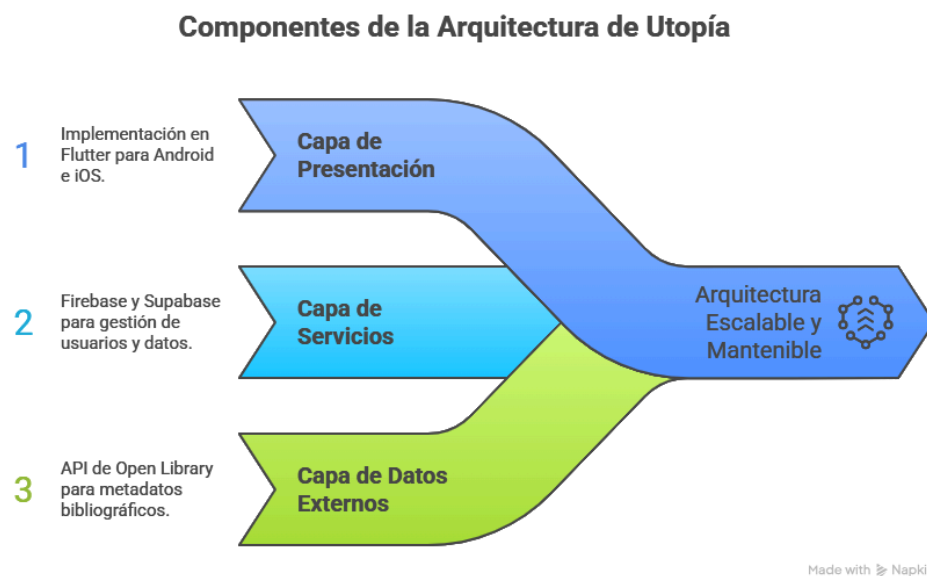


Figura 11. Diagrama de arquitectura de Utopía. Elaboración propia a partir de una plantilla Napkin

Modelo de datos

Relación de Intercambio (Economía Circular)

El modelo de datos de Utopía en Supabase está compuesto por dos tablas principales con relación funcional que implementa el núcleo de la economía circular de la plataforma.

La tabla `perfiles` almacena la información de cada usuario registrado. Sus campos principales son:

- `id` (clave primaria vinculada al UUID de Firebase Authentication),
- `user_name` (texto),
- `bio` (texto),
- `avatar_url` (texto),
- `books_exchanged` (entero, contador automático de intercambios completados),
- `is_eco_hero` (booleano, asignado automáticamente por trigger),
- `email`(texto),
- `created_at` (marca de tiempo),
- `last_login` (marca de tiempo).

La tabla `books_circulation` actúa como el pasaporte digital de cada ejemplar físico dentro del sistema. Sus campos son:

- `id` (UUID, clave primaria),
- `original_owner_id` (texto, clave foránea hacia perfiles),
- `current_owner_id` (texto, clave foránea hacia perfiles),
- `title` (texto),

- ISBN (texto) y
- status (texto, controla la disponibilidad del libro para intercambio).

El campo `original_owner_id` es inmutable y registra permanentemente al usuario que introdujo el libro en el ecosistema, mientras que `current_owner_id` es dinámico y se actualiza con cada intercambio completado. Esta separación permite al sistema rastrear toda la cadena de custodia del libro y otorgar logros diferenciados tanto al primer propietario como a los propietarios sucesivos.

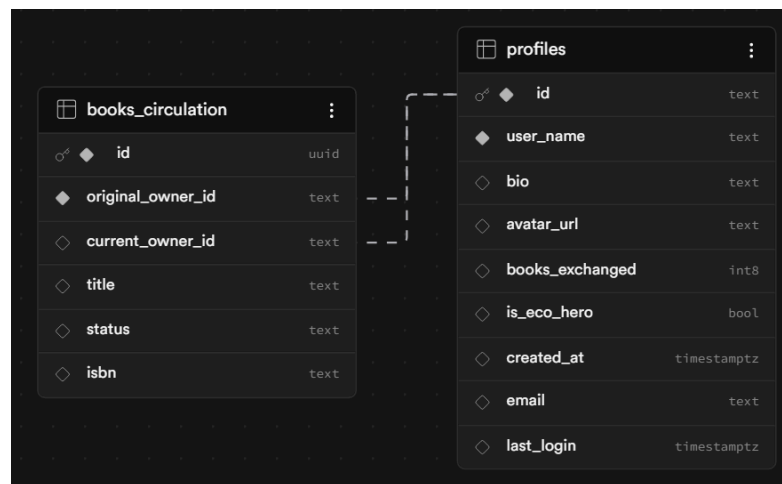


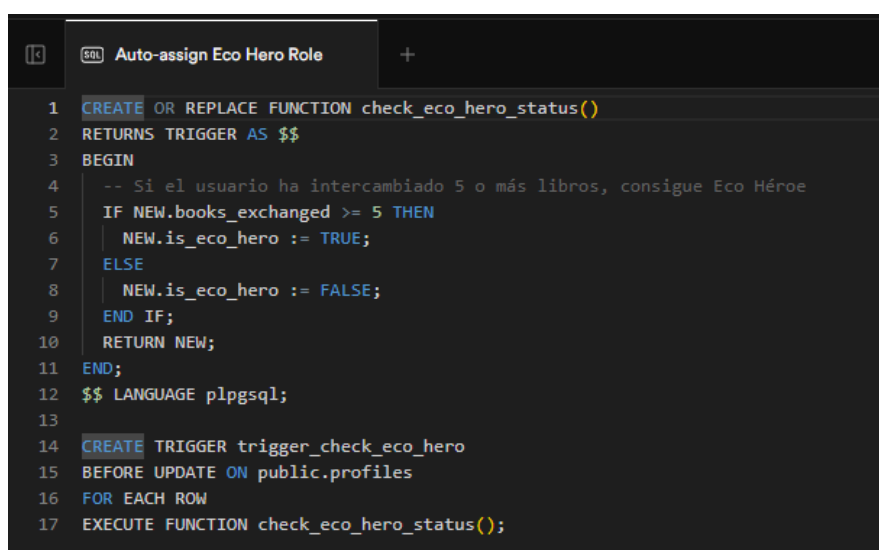
Figura 12. Diagrama del modelo de datos de Supabase. Elaboración propia.

Lógica de backend: triggers en PostgreSQL

Sistema de Gamificación y Logros de Perfil

Una de las decisiones técnicas más relevantes del proyecto es la implementación de lógica de negocio directamente en la base de datos mediante triggers de PostgreSQL, en lugar de gestionar esta lógica en el cliente Flutter, esto hace que el sistema sea más seguro y eficiente.

La función `check_eco_hero_status` se ejecuta automáticamente antes de cada operación de actualización sobre la tabla `profiles`. Su lógica evalúa si el campo `books_exchanged` alcanza o supera el umbral de 5 intercambios y, en caso afirmativo, actualiza el campo `is_eco_hero` a `true` de forma transparente para el usuario. Esta aproximación garantiza que la asignación del logro es automática, consistente e imposible de manipular desde el cliente, ya que la validación ocurre en el servidor de base de datos antes de que los datos persistan.



```

1 CREATE OR REPLACE FUNCTION check_eco_hero_status()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Si el usuario ha intercambiado 5 o más libros, consigue Eco Héroe
5     IF NEW.books_exchanged >= 5 THEN
6         NEW.is_eco_hero := TRUE;
7     ELSE
8         NEW.is_eco_hero := FALSE;
9     END IF;
10    RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 CREATE TRIGGER trigger_check_eco_hero
15 BEFORE UPDATE ON public.profiles
16 FOR EACH ROW
17 EXECUTE FUNCTION check_eco_hero_status();

```

Figura 13. Código del trigger `check_eco_hero_status` en `plpgsql`. Elaboración propia

Sistema de autenticación con Firebase.

La gestión de identidad de Utopía se implementa mediante Firebase Authentication ([Google, s.f-f](#)) con proveedor de correo electrónico y contraseña, seleccionado por tres razones arquitectónicas: la integración en Flutter a través de SDK oficial, la gestión automática del ciclo de vida de tokens sin intervención del desarrollador, y su capacidad de actuar como fuente única de verdad de identidad

vinculable con Supabase mediante el UID del usuario. La implementación técnica detallada, incluye el componente AuthGate y la sincronización con Supabase, se desarrolla en el apartado del bloque 8 .

Integración con la API de Open Library

La API pública de Open Library ([Internet Archive, s.f.](https://openlibrary.org/)), desarrollada y mantenida por Internet Archive, constituye la capa de datos bibliográficos externos de Utopía. Su elección frente a otras alternativas comerciales responde a tres criterios fundamentales: acceso completamente gratuito y sin límite de peticiones para uso no comercial, ausencia de requisito de autenticación mediante API key, y una base de datos de más de veinte millones de registros bibliográficos con cobertura internacional.

La integración se produce en dos puntos diferenciados de la aplicación. El primero es el endpoint de tendencias semanales, consultado desde HomeScreen mediante una petición HTTP GET a

<https://openlibrary.org/trending/weekly.json?limit=12>, que devuelve los doce títulos más leídos globalmente durante la semana en curso, actualizando de forma dinámica en cada sesión. El segundo punto de integración es la API de portadas, que permite construir la URL de la imagen de cualquier libro únicamente a partir de su ISBN, siguiendo el patrón

<https://covers.openlibrary.org/b/isbn/{ISBN}-M.jpg>. Esta segunda integración no requiere ninguna petición HTTP adicional desde el repositorio, ya que la URL se construye en tiempo de renderizado dentro del componente BookCard.

Desde el punto de vista arquitectónico, la decisión de no almacenar las portadas en Supabase sino obtenerlas dinámicamente desde Open Library responde a un criterio de eficiencia: elimina la necesidad de gestionar un sistema de almacenamiento de imágenes propio, reduce el tamaño de la base de datos y garantiza que las portadas mostradas corresponden siempre a la edición registrada por ISBN, sin posibilidad de desincronización entre la imagen almacenada y el ejemplar real.

Tecnologías utilizadas

Las tecnologías seleccionadas para el desarrollo de Utopía responden a criterios de idoneidad técnica, coste cero para el proyecto y capacidad de escalabilidad futura. A continuación se describe cada herramienta y su función específica dentro del sistema.

Flutter y Dart ([Google, s.f.-a](#)) constituyen el núcleo del proyecto. Flutter es el framework de desarrollo multiplataforma creado por Google que permite compilar una única base de código en aplicaciones nativas para Android e iOS simultáneamente. Dart es el lenguaje de programación sobre el que Flutter está construido, con tipado estático y orientación a objetos. La elección de Flutter frente a alternativas como React Native responde a su motor de renderizado propio, que garantiza una interfaz de usuario fluida y visualmente consistente en todos los dispositivos sin depender de componentes nativos del sistema operativo.

Firebase Authentication ([Google, s.f.-f](#)) gestiona íntegramente el ciclo de identidad del usuario en Utopía: registro, inicio de sesión, cierre de sesión y mantenimiento automático de la sesión entre arranques de la aplicación. Su integración con Flutter a través del SDK oficial elimina la necesidad de implementar un servidor de autenticación propio, delegando en la infraestructura de Google la gestión segura de contraseñas y tokens

Supabase ([Supabase, Inc., s.f.](#)) actúa como el backend principal del proyecto. Ofrece una base de datos PostgreSQL en la nube con una API REST generada automáticamente, lo que permite realizar operaciones CRUD desde Flutter sin necesidad de desarrollar un servidor intermedio. Además de la persistencia de datos, Supabase aloja la lógica de gamificación del sistema mediante triggers nativos de PostgreSQL, garantizando que las reglas de negocio se ejecutan en el servidor y no pueden ser manipuladas desde el cliente.

La API pública de Open Library ([Internet Archive, s.f.](#)), desarrollada y mantenida por Internet Archive, proporciona los metadatos bibliográficos que Utopía consume en dos puntos diferenciados: el endpoint de tendencias semanales para la pantalla principal y la API de portadas para la carga automática de imágenes a partir del ISBN. Su uso es completamente gratuito, no requiere autenticación mediante API key y ofrece acceso a más de veinte millones de registros bibliográficos con cobertura internacional.

Visual Studio Code ([Microsoft, s.f.](#)) ha sido el entorno de desarrollo principal durante todo el proyecto. Con las extensiones oficiales de Flutter y Dart instaladas, proporciona autocompletado, análisis estático de código, ejecución en

dispositivo físico y hot reload, que permite visualizar los cambios en la interfaz en tiempo real sin necesidad de recompilar la aplicación completa.

Git y GitHub ([Software Freedom Conservancy, s.f.](#)) han gestionado el control de versiones del proyecto durante todo el ciclo de desarrollo. Git registra el historial completo de cambios del código, mientras que GitHub actúa como repositorio remoto. La configuración del archivo .gitignore garantiza que las credenciales sensibles (claves de Supabase y configuración de Firebase) no se incluyen en el repositorio público, siguiendo las buenas prácticas de seguridad en proyectos de software.

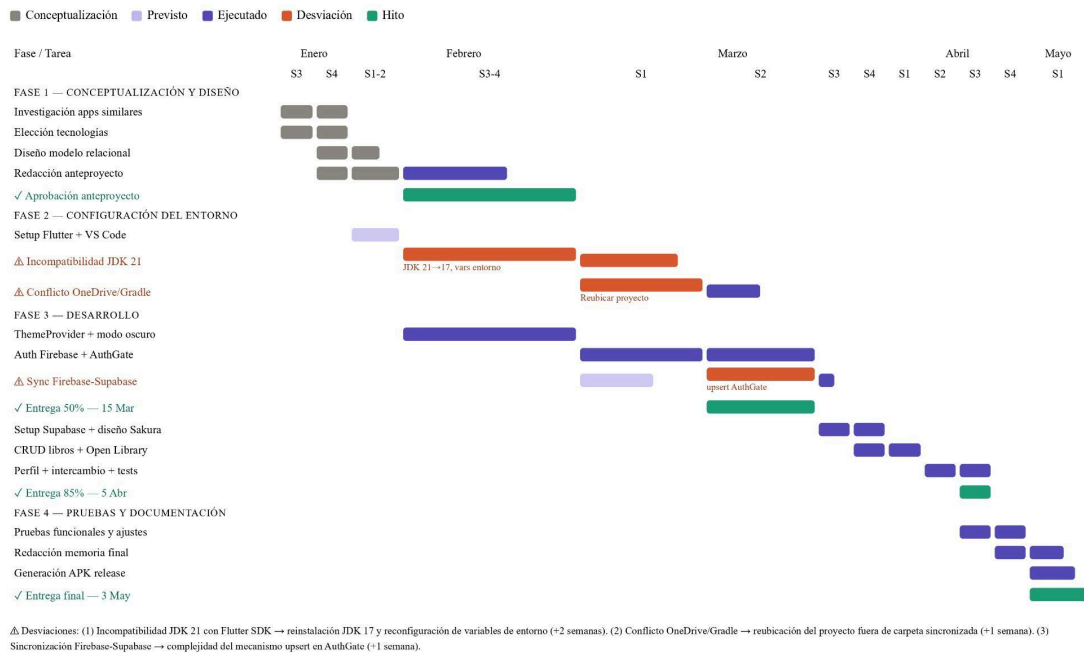
FlexColorScheme ([Ryden, s.f.](#)) es la librería de Flutter utilizada para la definición del sistema de temas visuales de la aplicación. Genera automáticamente una paleta de colores coherente y accesible tanto para el modo claro como para el modo oscuro a partir del esquema predefinido Sakura, garantizando que todos los componentes de la interfaz respetan el mismo lenguaje visual sin necesidad de definir colores individuales en cada widget.

Napkin.ai ([Napkin AI, s.f.](#)) es la herramienta de generación de diagramas mediante inteligencia artificial utilizada para crear las representaciones visuales de los requisitos RFTP incluidos en el Bloque 4 de esta memoria. Permite describir un flujo en lenguaje natural y obtener un diagrama estructurado de forma automática, agilizando la documentación técnica del proyecto.

Bloque 7 - Plan de trabajo

Planificación

Diagrama de Gantt



Planificación real + desviaciones.

La ejecución del proyecto siguió en líneas generales la estructura prevista, aunque se registraron tres desviaciones significativas respecto a la planificación inicial, todas ellas concentradas en la fase de configuración del entorno de desarrollo. A continuación se presenta la comparativa entre lo planificado y lo ejecutado, seguida del análisis de cada desviación.

La fase de conceptualización y redacción del anteproyecto se desarrolló según lo previsto entre finales de enero y el 22 de febrero. Durante este periodo se definió el alcance del proyecto, se investigaron las soluciones existentes y se establecieron los objetivos principales que articularían el desarrollo posterior.

La fase de configuración del entorno de desarrollo, sin embargo, experimentó la desviación más significativa del proyecto. Se estimaba resolverla en aproximadamente una semana, pero se extendió durante casi tres semanas debido a una cadena de incompatibilidades técnicas. El primer bloqueo fue la incompatibilidad entre la versión de Java JDK 21 instalada en el sistema y los requisitos del SDK de Flutter, que en ese momento exigía JDK 17. Fue necesario desinstalar la versión 21, instalar la 17 y reconfigurar las variables de entorno del sistema operativo para que las herramientas de compilación la reconocieran correctamente.

El segundo bloqueo fue la interferencia de OneDrive con la caché de Gradle: al tener la carpeta del proyecto sincronizada con OneDrive, el sistema de archivos generaba conflictos que corrompían los archivos de compilación de Android. La solución fue reubicar el proyecto fuera de la carpeta sincronizada y limpiar manualmente la caché de Gradle. Estas incidencias retrasaron el primer build funcional en dispositivo físico, aunque no impidieron el desarrollo de código durante ese periodo, ya que la aplicación podía probarse en el emulador mientras se resolvían los problemas de compilación nativa.

La tercera desviación, de menor impacto, fue el tiempo dedicado a la sincronización entre Firebase y Supabase en el componente AuthGate. La coexistencia de dos servicios de backend con sistemas de identidad distintos requirió más investigación de la prevista para implementar el mecanismo de upsert que garantiza la creación automática del perfil en Supabase cada vez que un usuario se autentica en Firebase sin generar duplicados.

En conjunto, las desviaciones acumularon un retraso aproximado de dos semanas respecto a la planificación inicial, que fue absorbido reduciendo el margen de tiempo reservado para pruebas y documentación. Las entregas parciales se realizaron en las fechas establecidas: la primera entrega al 50% el 15 de marzo y la segunda al 85% el 5 de abril, lo que confirma que el proyecto mantuvo un ritmo de desarrollo compatible con los hitos fijados por el centro a pesar de las incidencias técnicas iniciales.

Bloque 8 - Desarrollo de la solución

Estructura del proyecto Flutter

El proyecto Flutter sigue una organización de carpetas basada en Clean Architecture ([Martin, 2018, p.203](#)) con enfoque Feature-First, y su desarrollo se ha ejecutado siguiendo una metodología iterativa e incremental , implementando cada funcionalidad como un módulo independiente.

La estructura principal es la siguiente:

lib/core/

carpeta transversal que centraliza los elementos compartidos por todos los módulos de la aplicación, organizada en dos subcarpetas. La primera es themes/, que contiene los tres archivos que definen y gestionan el sistema visual global:

light_mode.dart y dark_mode.dart definen las paletas de colores para cada modo usando el esquema Sakura de FlexColorScheme, y theme_provider.dart gestiona el estado del tema activo mediante el patrón Provider con ChangeNotifier, permitiendo el cambio dinámico entre modos desde cualquier punto de la aplicación. La segunda es supabase_conf.dart/, que contiene configuracion.dart con las credenciales de conexión al proyecto Supabase (URL y clave anónima), y configuracion.example.dart, una plantilla sin credenciales reales que sirve de referencia para que cualquier desarrollador que clone el repositorio sepa qué variables de entorno debe configurar sin exponer datos sensibles.

lib/features/auth/

módulo de autenticación organizado en tres capas siguiendo Clean Architecture: la capa data contiene services/auth_service.dart, que gestiona todas las operaciones con Firebase Authentication (registro, login, cierre de sesión y sincronización con Supabase); la capa domain define los contratos de datos del sistema de identidad mediante classes/profile.dart, modelo del perfil de usuario, y classes/rol.dart, que establece la estructura de roles prevista para futuras versiones; y la capa presentation se divide a su vez en components/drawers_one.dart, el menú lateral con navegación y switch de tema, y seis pantallas en screens/: auth_gate.dart (portero reactivo de sesión), login_screen.dart, register_screen.dart, home_screen.dart, profile_screen.dart, public_profile_screen.dart (perfil público de otros usuarios) y exchange_screen.dart (pantalla de intercambio de libros).

lib/features/books/

módulo de gestión bibliográfica organizado en tres capas siguiendo Clean

Architecture:

la capa data contiene `book_repository.dart`, que centraliza todas las operaciones CRUD contra Supabase; la capa domain define `book_model.dart`, el contrato de datos entre la interfaz y la base de datos; y la capa presentation separa la lógica visual en `screens/books_screen.dart`, pantalla principal de la biblioteca personal, y `widgets/book_card.dart`, componente reutilizable que renderiza cada libro con su portada desde Open Library.

lib/main.dart/

punto de entrada de la aplicación, estructurado en dos bloques diferenciados. El primero es la función `main()`, declarada como `async` para permitir la inicialización secuencial de los servicios externos antes de arrancar la interfaz:

`WidgetsFlutterBinding.ensureInitialized()` garantiza que Flutter está completamente configurado antes de ejecutar código asíncrono;

`Firebase.initializeApp()` inicializa Firebase con las opciones específicas de plataforma definidas en `firebase_options.dart`; y `Supabase.initialize()` conecta la aplicación con el proyecto de Supabase mediante las credenciales almacenadas en `SupabaseConfig`. Una vez completadas las tres inicializaciones, `runApp()` lanza la aplicación envuelta en un `ChangeNotifierProvider` que instancia el `ThemeProvider` globalmente, haciendo el sistema de temas accesible desde cualquier punto del árbol de widgets. El segundo bloque es la clase `MainApp`, un `StatelessWidget` que configura el `MaterialApp` con tres decisiones clave:

`debugShowCheckedModeBanner: false` para eliminar la etiqueta visual de

depuración, home: AuthGate() como pantalla inicial que decide el flujo de navegación según el estado de sesión, y theme:

Provider.of<ThemeProvider>(context).themeData para aplicar el tema activo de forma reactiva en toda la interfaz.

lib/firebase_options.dart

archivo generado automáticamente por la CLI de FlutterFire que centraliza la configuración específica de Firebase para cada plataforma soportada por el proyecto. La clase DefaultFirebaseOptions expone un getter currentPlatform que evalúa en tiempo de ejecución la plataforma activa mediante defaultTargetPlatform y devuelve el conjunto de credenciales correspondiente: apiKey, appId, messagingSenderId, projectId y storageBucket. El archivo incluye configuración para cinco plataformas: Android, iOS, macOS, Web y Windows. La plataforma Linux lanza un UnsupportedError explícito ya que no ha sido configurada. Este archivo se incluye en el repositorio para que cualquier desarrollador que clone el proyecto pueda compilar y ejecutar la aplicación sin necesidad de reconfigurar Firebase manualmente.

En cuanto al entorno de desarrollo, se utilizó **JDK 17** como versión del kit de desarrollo Java, reubicando el SDK en la raíz del sistema para optimizar los tiempos de compilación y evitar conflictos de permisos habituales en entornos Windows. El control de versiones se gestiona mediante **Git**, con un archivo .gitignore configurado para excluir del repositorio los archivos de compilación, credenciales sensibles y otros de sistema operativo como .DS_Store, directorios de caché de Gradle y el directorio build/. Esta práctica garantiza que las claves de

API y las configuraciones de Firebase no queden expuestas en el repositorio público.

Implementación funcionalidades

El desarrollo de Utopía se ha estructurado siguiendo un enfoque iterativo e incremental ([Pressman & Maxim, 2021, p.38](#)), implementando cada funcionalidad como un módulo independiente dentro de la arquitectura Clean Architecture definida en el apartado anterior. A continuación se detalla la implementación técnica de cada una de las funcionalidades principales del sistema, relacionando cada decisión de diseño con los requisitos funcionales y no funcionales establecidos en el análisis previo.

Sistema de temas dinámicos (modo claro/oscuro)

En respuesta al requisito no funcional **RNF-01**, que exige una interfaz intuitiva con soporte para modo oscuro y claro, se ha implementado un sistema de gestión de temas global mediante el patrón **Provider** con la clase ThemeProvider.

La clase ThemeProvider extiende ChangeNotifier y actúa como única fuente de verdad sobre el estado visual de la aplicación. Su implementación se apoya en tres decisiones técnicas clave.

La primera es la selección del esquema de color. Se ha utilizado la librería FlexColorScheme con el esquema **FlexScheme.sakura**, que genera automáticamente una paleta de colores coherente y accesible tanto para el modo

claro como para el oscuro, garantizando que todos los componentes de la interfaz respetan el mismo lenguaje visual sin necesidad de definir colores individuales en cada widget.

La segunda es la gestión del estado global. El método `toggleTheme()` evalúa el estado actual mediante el getter `isDarkMode` y conmuta entre los dos temas definidos en `light_mode.dart` y `dark_mode.dart`, notificando automáticamente a todos los widgets suscritos mediante `notifyListeners()`. Esto garantiza que el cambio de apariencia se propague de forma instantánea a todo el árbol de widgets sin necesidad de reconstruir la interfaz completa.

La tercera es la accesibilidad desde cualquier punto de la app. La elección de Provider ([Google, s.f.-b](#)) como solución de gestión de estado responde a su integración nativa con el ecosistema Flutter, la eliminación del problema de prop-drilling al proporcionar acceso al tema desde cualquier punto del árbol de widgets, y su bajo coste de rendimiento frente a soluciones más complejas como Bloc o Riverpod, adecuado para el alcance actual del proyecto.

El punto de entrada de la aplicación, `main.dart`, envuelve el árbol completo con un `ChangeNotifierProvider` que instancia el `ThemeProvider`, haciendo el tema accesible de forma global:

La activación del tema por parte del usuario se realiza mediante un `Switch` integrado en el componente `DrawersOne`, que consume el `ThemeProvider` a través de `Provider.of<ThemeProvider>(context)` y llama al método `toggleTheme()` en cada interacción. El icono del switch se actualiza dinámicamente entre `Icons.dark_mode` e `Icons.light_mode` según el estado activo, ofreciendo retroalimentación visual inmediata al usuario.

El patrón de uso del sistema de temas es consistente en todos los widgets de la aplicación. En lugar de definir colores absolutos, cada componente solicita el esquema de colores activo mediante `Theme.of(context).colorScheme` y utiliza los roles semánticos definidos por Material 3, como `colorScheme.primary`, `colorScheme.surface` o `colorScheme.error`. Esta decisión arquitectónica garantiza que cualquier futuro cambio de paleta de colores se apliquen de forma automática a toda la interfaz sin modificar ningún widget individualmente.

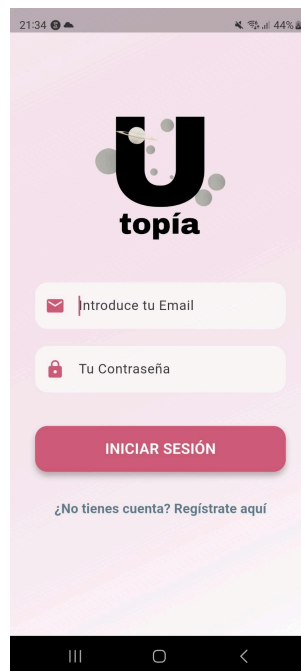


Figura 14. Login de Utopía en modo claro con esquema Sakura.

Elaboración propia.

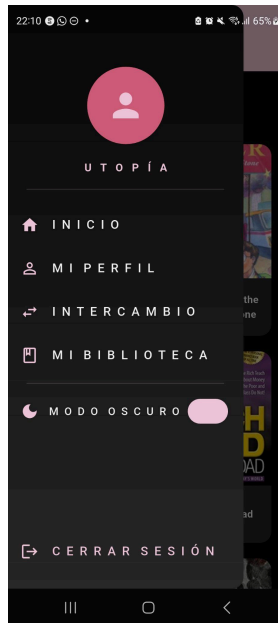


Figura 15. *Drawer de Utopía en modo oscuro con esquema Sakura.*

Elaboración propia.

Seguridad reactiva: AuthGate

En respuesta al requisito funcional **RF-01** y a la decisión de diseño establecida en el apartado Bloque 6, se detalla a continuación la implementación técnica del sistema de autenticación.

La gestión del flujo de navegación según el estado de autenticación se implementa mediante el componente AuthGate ([Google, s.f.-d](#)), un StatelessWidget que encapsula un `StreamBuilder<User?>` suscrito al stream `FirebaseAuth.instance.authStateChanges()`.

Este componente actúa como portero reactivo del árbol de widgets, evaluando tres estados posibles de forma automática y sin intervención del desarrollador ni del usuario. En primer lugar, si la conexión está en espera, muestra un

CircularProgressIndicator para evitar parpadeos en la interfaz durante la inicialización. En segundo lugar, si existe un usuario autenticado, ejecuta la sincronización con Supabase y redirige automáticamente a HomeScreen. En tercer lugar, si no hay sesión activa, redirige a LoginScreen.

Una decisión técnica relevante de este componente es la sincronización automática entre Firebase y Supabase. Dado que la autenticación se gestiona íntegramente mediante Firebase Authentication, es necesario garantizar que cada usuario autenticado tenga su perfil correspondiente en la tabla profiles de Supabase. Para ello, el método `_syncUserWithSupabase()` ejecuta una operación upsert cada vez que se detecta una sesión activa, insertando el perfil si no existe o manteniéndolo intacto si ya fue creado previamente. Esta operación es idempotente, lo que significa que puede ejecutarse múltiples veces sin generar duplicados ni errores.

Al estar basado en un stream reactivo, cualquier cambio de estado — ya sea un inicio de sesión, cierre de sesión o expiración de token — se propaga instantáneamente sin necesidad de gestión manual del ciclo de vida. Esta arquitectura elimina la posibilidad de que un usuario no autenticado acceda a pantallas protegidas, reforzando la seguridad de la aplicación a nivel de interfaz.

La decisión de separar la lógica de autenticación en un componente dedicado, en lugar de gestionarla directamente en `main.dart`, responde a los principios de Single Responsibility de Clean Architecture: el AuthGate tiene una única responsabilidad, decidir qué pantalla mostrar según el estado de autenticación, manteniendo el resto del árbol de widgets ajeno a esta lógica.

CRUD de libros con Supabase

En respuesta al requisito funcional **RF-03**, que establece la necesidad de gestionar el catálogo personal de libros mediante operaciones de creación, lectura, actualización y eliminación, se ha implementado un módulo completo de gestión bibliográfica estructurado en tres capas siguiendo los principios de Clean Architecture.

Capa Domain — BookModel

La clase BookModel actúa como contrato de datos entre la interfaz de usuario y la base de datos. Define los campos del libro y encapsula la lógica de serialización mediante dos métodos clave. El método `fromMap()` convierte la respuesta JSON de Supabase en un objeto Dart tipado, mientras que `toMap()` realiza la operación inversa para las operaciones de inserción. Esta separación garantiza que cualquier cambio en la estructura de la base de datos solo afecte a esta clase, sin impacto en el resto del sistema.

Capa Data — BookRepository

El repositorio centraliza toda la lógica de acceso a Supabase, exponiendo cuatro métodos que implementan las operaciones CRUD completas. Una decisión técnica relevante es el uso de `FirebaseAuth.instance.currentUser?.uid` para identificar al usuario en lugar del sistema de autenticación propio de Supabase. Esto es necesario porque la autenticación de la aplicación se delega íntegramente

a Firebase, por lo que el UID de Firebase actúa como clave de vinculación entre ambos servicios.

Capa Presentation — BooksScreen

La pantalla de gestión de la biblioteca personal implementa el patrón **FutureBuilder** ([Google, s.f.-e](#)) para gestionar los estados asíncronos de la consulta a Supabase. Este patrón permite distinguir tres estados de la interfaz de forma declarativa: estado de carga con `CircularProgressIndicator`, estado vacío con mensaje de bienvenida al usuario, y estado con datos renderizando la lista de libros mediante `ListView.builder`.

La operación de creación se implementa mediante un `AlertDialog` que solicita al usuario el título y el ISBN del libro. El campo ISBN es opcional, pero cuando se proporciona, activa automáticamente la carga de la portada real desde `Open Library` en el componente `BookCard`, conectando así el CRUD con la integración de la API externa definida en el **RF-02**.

La operación de eliminación incorpora un diálogo de confirmación antes de ejecutar el borrado en Supabase, previniendo eliminaciones accidentales. La operación de actualización de estado implementa el requisito **RF-05**, permitiendo al usuario alternar el libro entre los estados disponible y no disponible para intercambio mediante un único botón en la tarjeta del libro.

Integración Open Library

En respuesta al requisito funcional RF-02, que establece la necesidad de automatizar la obtención de datos bibliográficos mediante una API externa, se ha integrado la API pública de Open Library en dos puntos diferenciados de la aplicación: la pantalla principal HomeScreen y el componente BookCard.

HomeScreen - Dos secciones

La pantalla principal consume el endpoint de tendencias semanales de Open Library, que devuelve los títulos más leídos y valorados globalmente durante la semana en curso. Esta decisión sustituye una consulta estática por categoría, aportando contenido dinámico y actualizado en cada sesión:

La respuesta JSON de la API incluye los campos title, authors y cover_id, que se utilizan para construir dinámicamente cada tarjeta del grid. La URL de la portada se genera concatenando el identificador de portada con la base de la API de imágenes de Open Library:

La doble comprobación de cover_id y cover_i responde a una inconsistencia detectada durante el desarrollo en la estructura del JSON devuelto por el endpoint de tendencias, que utiliza indistintamente ambos campos según la versión del registro bibliográfico. Esta gestión defensiva garantiza que se recupere la portada siempre que el dato esté disponible bajo cualquiera de las dos claves.

Adicionalmente, la pantalla principal se rediseñó adoptando una estructura de dos secciones mediante CustomScrollView. La primera sección muestra en scroll horizontal los libros disponibles de otros usuarios desde Supabase. La segunda

mantiene el grid de tendencias de Open Library, optimizado a tres columnas para que la portada sea el elemento visual predominante.

Carga automática de portadas por ISBN — BookCard

El segundo punto de integración con Open Library se produce en el componente BookCard. Cuando un libro del catálogo personal dispone de ISBN, la portada se carga automáticamente sin ninguna llamada adicional a la API, construyendo la URL directamente a partir del ISBN almacenado en Supabase.

Esta aproximación es especialmente eficiente porque elimina la necesidad de realizar peticiones HTTP adicionales desde el repositorio: la URL se construye en tiempo de renderizado y el widget Image.network de Flutter gestiona la caché de forma nativa, evitando descargas repetidas de la misma portada.

Cuando el ISBN no está disponible o la imagen no puede cargarse, el componente muestra un placeholder con el icono de libro y los colores del esquema Sakura, garantizando una experiencia visual coherente en todos los casos mediante el parámetro errorHandler de Image.network.

Perfil de usuario real

En respuesta al requisito funcional RF-01 y al objetivo específico O.E.3, que establecen la necesidad de garantizar la seguridad y persistencia de datos del usuario, se ha implementado una pantalla de perfil que combina datos de dos fuentes externas: Firebase Authentication para los datos de identidad y Supabase para los datos de actividad dentro de la plataforma.

Arquitectura de datos del perfil

El perfil de usuario se construye a partir de la combinación de dos capas de información complementarias. La primera capa proviene de Firebase Authentication, que proporciona los datos de identidad básicos del usuario: el correo electrónico verificado y el UID único que actúa como clave de vinculación entre ambos servicios. La segunda capa proviene de la tabla profiles de Supabase, que almacena los datos de actividad dentro de la plataforma: el nombre de usuario, la biografía, el contador de libros intercambiados y el estado del logro Eco Héroe.

La carga del perfil se realiza mediante el método `_loadProfile()`, que ejecuta una consulta a Supabase filtrando por el UID de Firebase. Esta vinculación entre ambos servicios es posible gracias a la sincronización realizada por el componente AuthGate en el momento del inicio de sesión, que garantiza que cada usuario autenticado en Firebase tiene su registro correspondiente en la tabla profiles de Supabase:

Sistema de logros — Badge Eco Héroe

Una de las funcionalidades más relevantes del perfil es la visualización del logro Eco Héroe, que se activa automáticamente cuando el usuario completa cinco o más intercambios de libros. Este logro no se gestiona desde el cliente Flutter, sino que es asignado directamente por el trigger `check_eco_hero_status` ([PostgreSQL, 2024](#)) de PostgreSQL en Supabase. La pantalla de perfil únicamente lee el campo booleano `is_eco_hero` de la tabla profiles y renderiza el badge condicionalmente:

Avatar generado dinámicamente

En ausencia de una imagen de perfil subida por el usuario, el avatar se genera dinámicamente a partir de la inicial del nombre de usuario. Esta decisión evita la necesidad de gestionar un sistema de subida de imágenes en esta fase del proyecto, manteniendo la interfaz visualmente coherente y personalizada para cada usuario:

Tests unitarios

En respuesta a la necesidad de verificar la correcta serialización de los modelos de dominio, se han implementado tests unitarios sobre las clases BookModel y Profile, validando los métodos fromMap() y toMap() que conectan los objetos Dart con la base de datos Supabase.

Los tests se organizan en dos archivos dentro del directorio test/ del proyecto: book_model_test.dart y profile_test.dart. Cada archivo contiene tres casos de prueba que verifican, respectivamente, la conversión correcta de un Map completo a objeto Dart, la conversión inversa de objeto a Map para operaciones de inserción, y el manejo defensivo de valores nulos en campos opcionales.

La ejecución de los tests se realiza mediante el comando flutter test, que devuelve el resultado de forma automática. En la fase de entrega del proyecto, todos los tests pasan correctamente, confirmando la integridad de la capa de dominio independientemente de la conexión a Supabase. La figura 16 muestra el resultado de la ejecución de los tests mediante el comando flutter test, confirmando que los 6 casos de prueba pasan correctamente en el entorno de desarrollo.


```
PS C:\FLUTTER_APP_1\flutter_application_1> flutter test
00:06 +6: All tests passed!
PS C:\FLUTTER_APP_1\flutter_application_1> 
```

Figura 16. Resultado de la ejecución de flutter test con 6 tests unitarios superados.

Elaboración propia.

Triggers PostgreSQL (gamificación)

La lógica de gamificación de Utopía se implementa íntegramente en el servidor de base de datos mediante un trigger de PostgreSQL, siguiendo el principio de que las reglas de negocio críticas deben residir en la capa de datos y no en el cliente. Esta decisión arquitectónica garantiza que el logro Eco Héroe no puede ser manipulado desde la aplicación, ya que su asignación ocurre antes de que cualquier dato persista en la base de datos.

La solución se compone de dos elementos. El primero es la función `check_eco_hero_status()`, definida en `plpgsql`, que evalúa el valor del campo `books_exchanged` del registro que está siendo actualizado. Si dicho valor alcanza o supera el umbral de 5 intercambios completados, la función asigna `true` al campo `is_eco_hero` del mismo registro. En caso contrario, el campo se mantiene en `false`, lo que permite revertir el logro si el contador disminuyera en una versión futura del sistema.

El segundo elemento es el trigger `trigger_check_eco_hero`, que vincula la función anterior a la tabla `profiles` mediante una instrucción `BEFORE UPDATE FOR EACH ROW`. Esto garantiza que la función se ejecute automáticamente antes de cada actualización de cualquier fila de la tabla, sin necesidad de llamadas explícitas desde el cliente Flutter.

Para verificar el funcionamiento del sistema, se realizó una prueba directa sobre la base de datos: se accedió a la tabla profiles en el panel de Supabase y se modificó manualmente el campo books_exchanged de un usuario de prueba, asignándole el valor 6. El sistema actualizó automáticamente el campo is_eco_hero a true sin ninguna intervención adicional, demostrando que el trigger responde correctamente.

Bloque 9 - Despliegue, pruebas e instalación.

Despliegue e instalación

Entorno de desarrollo y ejecución en dispositivo físico

Durante la fase de desarrollo, la aplicación se ejecuta en modo debug sobre un dispositivo Android físico conectado mediante cable USB. Este método, denominado *sideloading* en modo depuración, requiere activar las opciones de desarrollador en el dispositivo y habilitar la depuración USB. Una vez configurado, el comando flutter run detecta el dispositivo automáticamente y despliega la aplicación compilada directamente sobre él, permitiendo el uso de hot reload para aplicar cambios en tiempo real sin necesidad de reinstalar.

Las dependencias del proyecto se gestionan mediante el archivo pubspec.yaml. Para instalar el entorno en una máquina nueva, los pasos necesarios son los siguientes: instalar Flutter SDK en su versión estable, clonar el repositorio desde Git, ejecutar flutter pub get para descargar todas las dependencias declaradas, y

configurar el archivo configuracion.dart con las variables de entorno de Supabase.

Las credenciales de Firebase se gestionan automáticamente mediante el archivo google-services.json incluido en el directorio android/app/.

Generación del APK para instalación independiente

Para distribuir la aplicación sin necesidad de cable ni entorno de desarrollo, Flutter permite generar un archivo APK mediante el comando flutter build apk --release. El archivo resultante se genera en la ruta build/app/outputs/flutter-apk/app-release.apk y puede instalarse en cualquier dispositivo Android habilitando la opción Instalar aplicaciones de fuentes desconocidas en los ajustes de seguridad del sistema.

Los requisitos mínimos del dispositivo para la instalación y ejecución de la aplicación son los siguientes: Android 5.0 (API nivel 21) o superior, 2 GB de RAM recomendados, 50 MB de espacio de almacenamiento libre y conexión a internet activa para la sincronización con Firebase Authentication y Supabase.

Es importante señalar que el APK generado en esta fase utiliza la firma de debug, lo que lo hace apto para pruebas y validación pero no para distribución en Google Play Store. La publicación en producción requeriría configurar un keystore de firma propio, tarea planificada como trabajo futuro tal como se recoge en el apartado 10.

Row Level Security (RLS) — Estado actual y plan de activación

Durante la fase de desarrollo, la política de Row Level Security de Supabase se mantiene deshabilitada en las tablas profiles y books_circulation, decisión tomada

deliberadamente para agilizar las pruebas de integración entre Firebase y Supabase sin añadir fricción en la validación funcional del sistema.

No obstante, se ha diseñado el esquema de políticas RLS que se activará antes de cualquier despliegue en producción. Las políticas previstas son las siguientes:

Tabla profiles:

- Un usuario solo puede leer su propio perfil (SELECT WHERE id = auth.uid()).
- Un usuario solo puede actualizar su propio perfil (UPDATE WHERE id = auth.uid()).
- La inserción está reservada al proceso de sincronización del AuthGate mediante upsert.

Tabla books_circulation:

- Cualquier usuario autenticado puede leer los libros con status = 'disponible' de otros usuarios (necesario para la funcionalidad de intercambio del CU-05).
- Un usuario solo puede insertar, actualizar o eliminar los registros donde original_owner_id = auth.uid().

Esta separación garantiza que ningún usuario pueda acceder ni modificar registros ajenos, cumpliendo el requisito no funcional **RNF-05** definido en el Bloque 5.

La activación de RLS implica además adaptar el método `_syncUserWithSupabase()` del componente AuthGate para que opere con un token de sesión válido de Supabase en lugar del UID de Firebase directamente, ya que

auth.uid() en las políticas de Supabase hace referencia a su propio sistema de autenticación. Esta adaptación está identificada como tarea prioritaria para la siguiente iteración del sistema, junto con la configuración del keystore de firma del APK mencionada en el apartado anterior.

Pruebas funcionales de backend

Las pruebas de backend se realizaron directamente sobre la base de datos de Supabase, sin intervención de la interfaz de usuario, con el objetivo de verificar que la lógica de negocio implementada en el servidor funciona de forma autónoma e independiente del cliente Flutter.

Se validaron tres escenarios. El primero comprueba la integridad del trigger de gamificación: accediendo al editor SQL de Supabase, se modificó manualmente el campo books_exchanged de un registro de la tabla profiles asignándole el valor 6. Tras ejecutar la actualización, el sistema verificó automáticamente que el campo is_eco_hero cambió a true sin ninguna intervención adicional, confirmando que el trigger check_eco_hero_status ([PostgreSQL, 2024](#)) se ejecuta correctamente antes de cada operación UPDATE sobre la tabla.

El segundo escenario verifica la idempotencia de la sincronización de perfiles: se simuló múltiples inicios de sesión con el mismo usuario para comprobar que la operación upsert ejecutada por _syncUserWithSupabase() no genera registros duplicados en la tabla profiles. En todos los casos el sistema mantuvo un único registro por UID, confirmando el comportamiento esperado.

El tercer escenario valida la integridad referencial del catálogo: se insertaron, actualizaron y eliminaron registros en la tabla books_circulation verificando que todas las operaciones quedan vinculadas correctamente al UID del usuario propietario y que ningún registro huérfano persiste tras una eliminación.

Pruebas de autenticación

El flujo completo de autenticación fue validado mediante tres escenarios de prueba. El primero verifica el registro exitoso: al introducir un correo y contraseña válidos en RegisterScreen, el usuario aparece en la consola de Firebase Authentication y su perfil se inserta automáticamente en la tabla profiles de Supabase. El segundo verifica el inicio de sesión: al introducir credenciales correctas en LoginScreen, el componente AuthGate detecta la sesión activa y redirige a HomeScreen sin intervención manual. El tercero verifica el manejo de errores: al introducir credenciales incorrectas, la aplicación muestra un SnackBar con mensaje de error y el usuario permanece en LoginScreen sin que se produzca ninguna redirección.

Pruebas de integración con API externa

La integración con Open Library fue validada en dos escenarios. En el primero, al iniciar la aplicación con conexión a internet activa, HomeScreen carga y muestra correctamente el grid de portadas y títulos obtenidos del endpoint de tendencias semanales. En el segundo, al añadir un libro con ISBN conocido a la biblioteca personal, el componente BookCard construye la URL de portada automáticamente y la imagen real del libro aparece en la lista sin ninguna petición adicional al

repositorio. En ausencia de ISBN o ante un error de red, se muestra el placeholder con el esquema de color Sakura, confirmando que el manejo de errores funciona correctamente.

Bloque 10 - Evolución y trabajo futuro

El alcance de la versión actual de **Utopía** constituye un **MVP funcional** que cubre el ciclo completo de la economía circular desde el punto de vista de la visibilidad: el usuario puede publicar sus libros como disponibles y otros lectores pueden consultarlos para coordinar un encuentro físico. Sin embargo, existen funcionalidades planificadas que ampliarán el sistema en iteraciones futuras.

RF pendiente — Flujo completo de confirmación de intercambio

La siguiente iteración prioritaria es la implementación del handshake digital de intercambio. Cuando dos usuarios acuerden el traspaso de un ejemplar, el sistema deberá ejecutar automáticamente dos operaciones en cascada sobre Supabase: actualizar el campo `current_owner_id` en la tabla `books_circulation` para reflejar el nuevo propietario, e incrementar el contador `books_exchanged` en la tabla `profiles` de ambos usuarios implicados. Este incremento activaría el trigger `check_eco_hero_status` de forma orgánica, sin necesidad de intervención manual en la base de datos, cerrando así el ciclo completo de gamificación.

Chat interno en tiempo real para coordinación de intercambios

Actualmente, la gestión de la cita para el intercambio se realiza fuera de la plataforma. Para cerrar el ciclo de usuario dentro de la app, se propone la integración de un módulo de mensajería instantánea.

- **Implementación Técnica:** Se utilizarán las capacidades de Realtime de Supabase o Firebase Cloud Messaging (FCM) para permitir una comunicación bidireccional instantánea.
- **Funcionalidades:**
 - Notificaciones push para nuevos mensajes.
 - Confirmación de lectura y estado "en línea".
 - Integración de un sistema de "Acuerdo de Intercambio" dentro del chat que, al ser aceptado por ambas partes, actualice automáticamente el estado del libro en la base de datos.

Expansión del ecosistema: Roles para bibliotecas y editoriales

Para transformar Utopía en un eje central del sector literario, se propone la creación de perfiles verificados con funcionalidades específicas:

- **Bibliotecas Públicas:** Podrán digitalizar su catálogo disponible para préstamo local, permitiendo a los usuarios reservar libros desde la app y fomentando la afluencia a centros culturales.
- **Editoriales y Librerías:** Introducción de un modelo de "Libros Patrocinados" o secciones de novedades donde puedan interactuar con la comunidad, ofreciendo ejemplares para sorteos o preventas exclusivas.

- **Gestión de Permisos:** Se implementará un sistema de **RBAC** (Role-Based Access Control) en el backend para segmentar las vistas y acciones según el tipo de cuenta (Usuario, Biblioteca o Admin).

Geolocalización y Mapas

Se integrará la API de Google Maps ([Google, s.f.-h](#)) o Mapbox ([Mapbox, s.f.](#)) para visualizar en un mapa interactivo los libros disponibles en las cercanías del usuario. Esto reducirá la fricción en el intercambio físico y permitirá establecer "Puntos Utopía" (cafeterías o librerías colaboradoras) como lugares seguros de encuentro.

Funcionalidad futura	Complejidad estimada	Tiempo estimado
Flujo confirmación de intercambio	Media	1-2 semanas
Chat en tiempo real	Alta	3-4 semanas
Roles bibliotecas y editoriales	Alta	4-6 semanas
Geolocalización y mapas	Media	2-3 semanas

Tabla 9. Funcionalidades futuras y su estimación. Elaboración propia

Conclusión

El desarrollo de Utopía ha permitido consolidar una solución tecnológica que trasciende el concepto de una simple base de datos bibliográfica. A través de este proyecto, se ha demostrado que la tecnología puede actuar como un puente para recuperar experiencias físicas y humanas en lugar de sustituirlas. La implementación de un sistema de economía circular, apoyado en una arquitectura robusta de Flutter y Supabase, no solo resuelve el problema del acceso económico a la cultura, sino que dota al libro de una nueva vida social.

Desde una perspectiva técnica, el proyecto cumple con los objetivos de escalabilidad y seguridad planteados. La integración reactiva de servicios como Firebase y la automatización de la lógica de negocio mediante triggers en PostgreSQL garantizan un sistema eficiente donde el usuario es el centro. El uso de la gamificación (Badge Eco Héroe) ha resultado ser una herramienta clave para incentivar el compromiso con la sostenibilidad, transformando la gestión de inventarios en una experiencia motivadora.

En conclusión, este trabajo representa un equilibrio entre la ingeniería de software moderna y el diseño ético. Utopía no es solo un gestor de libros; es una infraestructura diseñada para crear comunidad, fomentar el intercambio sostenible y devolver el protagonismo al lector. Con las bases establecidas en este MVP, la plataforma queda preparada para evolucionar hacia un ecosistema cultural más amplio, integrando geolocalización y nuevos roles institucionales, manteniendo siempre su esencia de conexión humana a través de la literatura.

Bibliografía.

Google. (s.f.-a). Flutter Documentation.

<https://docs.flutter.dev/>

Consultado como referencia oficial del framework Flutter y el lenguaje Dart durante todo el desarrollo del proyecto, incluyendo la implementación de widgets, navegación, gestión de estado y compilación multiplataforma para Android e iOS.

Google. (s.f.-b). Flutter. Simple app state management.

<https://docs.flutter.dev/data-and-backend/state-mgmt/simple>

Aplicado en la justificación de la elección del patrón Provider frente a alternativas más complejas como Bloc o Riverpod, para la gestión del estado global del tema visual en ThemeProvider.

Google. (s.f.-c.). provider package.

<https://pub.dev/packages/provider>

Utilizado en la implementación del ThemeProvider con ChangeNotifier para la gestión global del estado del tema claro/oscuro, haciendo el tema accesible desde cualquier punto del árbol de widgets sin prop-drilling.

Google. (s.f.-d). StreamBuilder class. StreamBuilder class-widget.

<https://api.flutter.dev/flutter/widgets/StreamBuilder-class.html>

Aplicado en el componente AuthGate, que usa un StreamBuilder suscrito al stream authStateChanges() de Firebase para gestionar reactivamente el estado de autenticación y decidir qué pantalla mostrar en cada momento.

Google. (s.f.-e). FutureBuilder class.

<https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html>

Aplicado en la justificación de la elección del patrón Provider frente a alternativas más complejas como Bloc o Riverpod, para la gestión del estado global del tema visual en ThemeProvider.

Google. (s.f.-f). Firebase Authentication.

<https://firebase.google.com/docs/auth>

Utilizado para implementar el sistema completo de identidad de usuario: registro con correo y contraseña, inicio de sesión, cierre de sesión y mantenimiento automático de la sesión mediante el SDK oficial de Firebase para Flutter.

Google. (s.f.-g). Material Design 3.

<https://m3.material.io/>

Aplicado en la interfaz de usuario mediante la activación de useMaterial3: true en la configuración de temas, utilizando los roles semánticos de color de Material 3 como colorScheme.primary, colorScheme.surface y colorScheme.onSurface .

Google. (s.f.-h). google_maps_flutter package.

https://pub.dev/packages/google_maps_flutter

Identificado como tecnología candidata para la implementación futura de geolocalización descrita en el apartado Bloque 10, que permitiría visualizar en un mapa interactivo los libros disponibles en las cercanías del usuario y establecer puntos de encuentro para el intercambio físico.

Internet Archive. (s.f.). Retrieved s.f.,

<https://openlibrary.org/developers/api>

Integrado en dos puntos del proyecto: el endpoint de tendencias semanales para la pantalla principal HomeScreen, y la API de portadas para la carga automática de imágenes a partir del ISBN almacenado en Supabase.

Mapbox. (s.f.). mapbox_maps_flutter package.

https://pub.dev/packages/mapbox_maps_flutter

Identificado como alternativa a Google Maps para la funcionalidad de geolocalización planificada en el apartado Bloque 10, con mayor flexibilidad en la personalización visual de los mapas y modelo de precios más favorable para proyectos con bajo volumen de peticiones.

Martin, R. C. (2012, August 13). Clean Coder Blog. Clean Coder Blog.

<https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

Artículo original donde Robert C. Martin publicó por primera vez el concepto de Clean Architecture. Consultado junto al libro para comprender los principios fundacionales que justifican la separación de responsabilidades entre capas en la estructura del proyecto.

Martin, R. C. (2018). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall Press.

Aplicado en el diseño de la arquitectura Feature-First del proyecto, que organiza el código en módulos funcionales independientes con separación en capas data, domain y presentation, siguiendo el principio de Single Responsibility.

Microsoft. (s.f.). Microsoft. Visual Studio Code documentation.

<https://code.visualstudio.com/docs>

Utilizado como entorno de desarrollo principal durante todo el proyecto, con las extensiones oficiales de Flutter y Dart para edición de código, depuración, análisis estático y ejecución en dispositivo físico mediante hot reload.

Napkin AI. (s.f.). Visual AI for business storytelling.

<https://www.napkin.ai>

Utilizado para la generación de los diagramas RFTP incluidos en el Bloque 4 de la memoria, describiendo los flujos en lenguaje natural y obteniendo representaciones visuales estructuradas de forma automática.

PostgreSQL. (2024). Global Development Group. . CREATE TRIGGER — documentation.

<https://www.postgresql.org/docs/current/sql-createtrigger.html>

Aplicado en la implementación del trigger check_eco_hero_status sobre la tabla profiles, que evalúa automáticamente el campo books_exchanged y asigna el logro Eco Héroe cuando el usuario supera cinco intercambios completados.

Pressman, R. S., & Maxim, B. R. (2021). Ingeniería del software: un enfoque práctico (9.^a ed.). McGraw-Hill.

Consultado como referencia metodológica para justificar el enfoque de desarrollo iterativo e incremental adoptado en la planificación y ejecución del proyecto, descrito en el Bloque 7 de la memoria.

Ryden, M. (s.f.). *flex_color_scheme* package.

https://pub.dev/packages/flex_color_scheme

Aplicado en la definición del esquema de colores "Sakura" para los modos claro y oscuro de la aplicación, generando automáticamente una paleta coherente y accesible en todos los componentes de la interfaz sin necesidad de definir colores individuales en cada widget.

Software Freedom Conservancy. (s.f.). *Git documentation*.

<https://git-scm.com/doc>

Utilizado para el control de versiones del proyecto durante todo el ciclo de desarrollo, gestionando el historial de cambios del código fuente y garantizando mediante el archivo .gitignore que las credenciales sensibles no se incluyen en el repositorio.

Supabase, Inc. (s.f.). *Supabase documentation*.

<https://supabase.com/docs>

Utilizado como backend principal del proyecto y documentado en el apartado de la memoria como tarea prioritaria antes del despliegue en producción, para garantizar que cada usuario solo pueda acceder y modificar sus propios registros en las tablas profiles y books_circulation, cumpliendo el requisito RNF-05.